

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY



APPLE KNIGHT ADVENTURE

CATALOG OF 130 FEATURES AND ANALYSIS OF 56 LEVELS/WAVES

Project Feature Catalog and Level Analysis Report

Course: Programming Systems – CS202

Class group: Group 55

Student ID	Full name
25125074	Nguyễn Anh Kiệt
25125037	Nguyễn Trọng Tiến

Ho Chi Minh City, August 2026

Abstract

This document reports a complete source-level audit of *Apple Knight Adventure*. The inspected scope includes the C++17/raylib game executable, authored levels and assets, the Survival3D subsystem, MapBuilder, Shop, the user-interface state system, persistence, and the optional Survival3D backend. The report establishes **130 independent project features**, identified as F001–F130. Separate from that feature count are **six 2D campaign levels**, identified as L2D-01–L2D-06, and **fifty Survival3D waves**, identified as W3D-01–W3D-50. The 56 levels and waves are analysed and catalogued independently; they are never used to inflate the feature total.

Every feature entry explains the mechanism, its relationships with other subsystems, its player-facing or engineering value, and its source evidence. The requested topics—PlayerInputs MovementCollision, EnemyBehavior, PowerUpsItems(potion), Level Completion across the 56-level/wave project, Sounds, OOD, DesignPatterns, AI, MultiplePlayers, and 3D Game—are mapped to explicit feature identifiers. The six runtime 2D levels are audited for dimensions, enemy and boss composition, items, checkpoints, portals, and authored interactions. All fifty 3D waves are described through their probability mix, threat budget, active cap, spawn interval, HP and damage scaling, and intended combat pressure; the five boss waves receive dedicated phase-level discussion.

The catalog covers the integrated two-player local co-op campaign, deterministic rule-based AI, the complete Shop purchase and unlock flow, MapBuilder, and the optional Survival3D result and leaderboard service. Feature claims are grounded in executable runtime paths, authored data, or buildable optional infrastructure.

Mandatory counting convention

130 independent features + 6 two-dimensional levels + 50 three-dimensional waves.

The report does not call a level or wave a feature. There are 186 catalog identifiers in total, while the number of independent project features remains 130.

Status Terminology

- **Connected to the runtime flow:** an executable call site or state transition was identified.
- **Implemented with an explicit boundary:** the core mechanism exists and its current data or service boundary is documented in the corresponding feature entry.
- **Optional infrastructure:** a genuine build or service capability that is not required for offline gameplay.

Contents

Abstract	i
Status Terminology	i
1 Audit Scope and Method	1
2 Mandatory Requirement Traceability	2
3 Level and Wave Completion Across the 56-Stage Project	3
4 Startup, Interface, and Player Configuration	4
5 Player Inputs, Movement, and Collision	7
6 Four 2D Character Skill Sets	13
7 Items, Level Completion, and 2D Feedback	23
8 Completion, Campaign Feedback, and Editor Entry	28
9 EnemyBehavior, AI, Build Systems, and MultiplePlayers	33
10 Survival3D Features	41
11 Sound, Persistence, OOD, and Design Patterns	53
12 Shop, Custom Levels, MapBuilder, UI, and Build	64
13 Six 2D Campaign Levels — A Separate Catalog	69
14 Fifty Survival3D Waves - Separate Level Catalog	75
Catalog Conclusion	106
A Feature Coverage Matrix	107

B Level and Wave Coverage Matrix	107
C Source-Evidence Index	108

List of Figures

1	Completion flows for all six campaign levels and fifty Survival3D waves . . .	3
---	---	---

List of Tables

1	Audited domains and resulting identifiers	1
2	Mapping of all user-mandated topics	2
3	Summary of the six runtime 2D levels	70
4	Director parameters for W3D-01–W3D-50	76
5	Coverage of all 130 independent features	107
6	The 56 content identifiers remain separate from the feature total	107
7	Principal evidence locations by subsystem	108

1 Audit Scope and Method

1.1 Inspected Project Surface

The audit covers the complete `AppleKnightAdventure/` tree: 97 headers, 87 C++ implementation files, and approximately 46,022 physical source lines. The asset catalog contains 315 PNG images, 178 JSON documents, 90 WAV samples, 24 GLB models, and 8 LDtk projects. In addition to the game executable, the repository contains the `backend/survival3d` service and an optional `AegisRiftServer` target. These figures establish the breadth of the audit; they are not counted as features.

Table 1: Audited domains and resulting identifiers

Domain	Principal components	Catalog range
2D campaign	Controllers, Player, skill sets, enemies, bosses, items, and six LDtk levels	F001–F075 and L2D-01–L2D-06
Survival3D	Controller/View, animation, IK, VFX, wave and balance configuration	F076–F100 and W3D-01–W3D-50
Cross-cutting systems	Audio, persistence, renderer, asset loading, OOD, and design patterns	F101–F120
Tools, UI, and build	Shop, custom content, MapBuilder, UI states, CMake, and the optional backend	F121–F130

1.2 Source-Audit Method

Each claim was traced from input or authored data, through the responsible controller and model, to a view, persistence operation, or externally observable state. A class that exists in the repository but has no call site in the principal runtime was not promoted to a completed runtime feature. Level files were inspected using the same level index selected by `GameController`; consequently, `tutorial.ldtk` is the active first campaign level, whereas `level1.lvl` is only a legacy fallback. A `Survival3D` mix is interpreted as a weighted selection probability and never as a guaranteed enemy count.

The resulting catalog uses three non-overlapping identifier ranges: F001–F130 for independent features, L2D-01–L2D-06 for authored two-dimensional campaign levels, and W3D-01–W3D-

50 for three-dimensional survival waves. This separation prevents content volume from inflating the feature count.

2 Mandatory Requirement Traceability

Table 2: Mapping of all user-mandated topics

Required topic	Related features	Verified content
PlayerInputsMovement Collision	F009–F020, F077–F080	Solo/P1/P2 input, movement, jump, sprint, dash, parry, cast lock, tile/FakeWall AABB collision, fall/respawn, and three-dimensional movement/camera interaction.
EnemyBehavior	F061–F067, F074, F091–F093	Three 2D enemy types, FSM and telegraphs, LOS and ledge avoidance, three bosses and navigation, three regular 3D archetypes, and five 3D bosses.
PowerUpsItems (potion)	F041–F042, F044, F046, F069–F072, F094	Coin, Apple, 50-HP Potion, Chest rewards, twelve boons, twenty-one cores, elemental reactions, and eighteen Survival3D upgrades.
LevelCompletion across 56 levels/waves	F050–F053, F093, F100; L2D-01–06; W3D-01–50	Six campaign finishes lead through scoring, persistence, results, and sequential unlocking; fifty survival waves progress through clear, reward, boss, and final RunVictory states.
Sounds	F057, F101–F105	Volume options, manifest-driven events, non-repeating samples and pitch, cooldown and layering, spatial audio, and contextual music.
OOD	F110–F111	MVC-like separation, inheritance, composition, smart-pointer ownership, and runtime polymorphism.

Required topic	Related features	Verified content
DesignPatterns	F110, F112–F119	MVC, Singleton, Factory, Adapter, Command/Composite, Object Pool, Quadtree, event callback, and producer–consumer queues.
AI	F061–F068, F091–F093	Rule-based FSMs, detection, LOS, ledge avoidance, footprint-aware BFS, pet policies, wave direction, and boss policies.
MultiplePlayers	F008, F010–F011, F020, F073	Two-player local co-op with separate input and HUD, shared camera and upgrades, target selection, and partner rescue.
3D Game	F076–F100 and W3D-01–W3D-50	Hero selection, movement, camera, skills, AI, director, upgrades, fixed step, pooling, animation/VFX, accessibility, run results, and all fifty waves.

3 Level and Wave Completion Across the 56-Stage Project

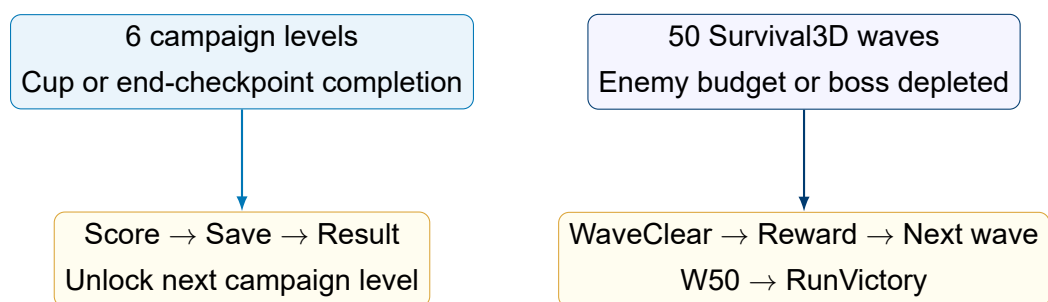


Figure 1: Completion flows for all six campaign levels and fifty Survival3D waves

The campaign and Survival3D modes use different completion models. A campaign map reaches its finish through an authored cup or end checkpoint and then evaluates score, stars, persistence, and the next-level unlock. Survival3D treats each enemy-budget or boss defeat as one wave completion, applies the inter-wave reward flow through W3D-49, and changes

W3D-50 into the final RunVictory and run-result sequence. Together these flows cover every one of the project's 56 playable level/wave identifiers.

4 Startup, Interface, and Player Configuration

4.1 F001 — Progress-Aware Startup Asset Loading

Subsystem: Startup and assets — **Status:** Integrated into runtime

Definition. The program does not load every image, atlas, sound, and data file in one blocking operation. AssetManager organizes a work list, decodes images on a worker thread, and returns GPU texture creation to the main thread. The loading screen can therefore update its percentage and status message while the game is being prepared.

Role and interactions. This mechanism is the entry point for almost every visual subsystem. It reduces the impression that the application has frozen while respecting the raylib/OpenGL requirement that GPU work be performed by the thread that owns the graphics context.

Source-code/data evidence. AppleKnightAdventure/src/main.cpp; AppleKnightAdventure/src/View/AssetManager.cpp.

4.2 F002 — Resizable Window, DPI Handling, and Adaptive Layout

Subsystem: Window framework and UI — **Status:** Integrated into runtime

Definition. WindowManager tracks framebuffer dimensions, the DPI scale, and resize events. Views read the current dimensions when laying out panels, buttons, and the camera instead of assuming one fixed resolution.

Role and interactions. Menus, the HUD, and gameplay remain usable when the application switches between windowed and fullscreen modes or runs on a display whose scaling is not 100 percent. The same capability also supports the Survival3D UI-scale option.

Source-code/data evidence. AppleKnightAdventure/src/Systems/WindowManager.cpp; AppleKnightAdventure/src/View/UIHelpers.cpp.

4.3 F003 — Main Menu and Feature Navigation

Subsystem: Out-of-game interface — **Status:** Integrated into runtime

Definition. MenuController routes the Main Menu to Continue, New Game, Level Select, Prepare, Shop, Custom Level, MapBuilder, Leaderboard, Achievement, Options, and Quit. MenuView is responsible for presentation, animation, and selection input, while the controller owns state transitions.

Role and interactions. The menu is the hub of the entire project, exposes the MVC boundary, and lets every tool or mode maintain its own lifecycle. A route that is unavailable can return an error state without damaging the main game loop.

Source-code/data evidence. AppleKnightAdventure/src/Controller/MenuController.cpp; AppleKnightAdventure/src/View/MenuView.cpp.

4.4 F004 — Level Selection and Sequential Unlocking

Subsystem: 2D campaign — **Status:** Integrated into runtime

Definition. Level Select reads progression from SaveManager, presents six level cards, and permits only unlocked levels to start. When a campaign level is completed, GameController records its result and unlocks the next level in sequence.

Role and interactions. This flow creates deliberate campaign progression: players can see the structure of all six levels, but must finish the preceding level before advancing. Best score, stars, and completion time are displayed on the corresponding level card.

Source-code/data evidence. AppleKnightAdventure/src/Controller/MenuController.cpp; AppleKnightAdventure/src/Model/SaveManager.cpp.

4.5 F005 — Prepare Screen and Loadout Configuration

Subsystem: Pre-level preparation — **Status:** Integrated into runtime

Definition. PrepareController and PrepareView collect the character class, pet, and player-count choices before GameState is created. These choices become explicit runtime configuration rather than forcing GameController to infer them from a sprite or stale save data.

Role and interactions. This screen separates tactical loadout decisions from combat and makes it faster to test each class or local co-op configuration. It also remembers the most recent selection so the next run does not always begin from default settings.

Source-code/data evidence. AppleKnightAdventure/src/Controller/PrepareController.cpp; AppleKnightAdventure/src/View/PrepareView.cpp.

4.6 F006 — Four 2D Character Classes with Distinct Skill Sets

Subsystem: 2D characters — **Status:** Integrated into runtime

Definition. Knight, Fighter, Magic Caster, and Ninja each provide a separate implementation of CharacterSkillSet. Player owns the selected skill set through composition, so changing class replaces the behavior of J/K/U/H, cooldowns, projectiles, and animations rather than changing only the artwork.

Role and interactions. The design supports four recognizable styles: Knight is balanced and close-range, Fighter concentrates damage, Magic Caster attacks at range through elemental effects, and Ninja emphasizes mobility and burst damage. It is also a central example of runtime polymorphism in the project.

Source-code/data evidence. AppleKnightAdventure/include/Model/CharacterSkillSet.h; AppleKnightAdventure/src/Model/Player.cpp.

4.7 F007 — Companion Pet Selection

Subsystem: Pets and loadouts — **Status:** Integrated into runtime

Definition. Prepare and Shop allow the player to select Skull, Ghost, Baby Dragon, or Fairy. GameController creates the selected pet in single-player mode, and the pet retains the references it needs to query the player and world when updating its position, target, or item behavior.

Role and interactions. Pets add another loadout dimension without replacing the character class. Disabling pets in local co-op is a balancing and entity-complexity decision, not an asset-loading defect.

Source-code/data evidence. AppleKnightAdventure/src/Controller/PrepareController.cpp; AppleKnightAdventure/src/Model/Pet.cpp.

4.8 F008 — Solo and Local Co-op Switching

Subsystem: MultiplePlayers — **Status:** Integrated into runtime

Definition. PrepareView switches between one-player and two-player configurations. ConfigureLocalCoop decides whether P2 is created, places P2 beside P1 at spawn, and activates the corresponding branches for input, HUD, camera, AI targeting, and teammate revival.

Role and interactions. A shared GameState lets both players share persistent coin rewards, encounter progress, portals, and upgrades, while health, cooldowns, and input remain separate. Teammate-aware respawn and revival rules prevent one defeat from immediately ending the shared encounter.

Source-code/data evidence. AppleKnightAdventure/src/Controller/PrepareController.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

5 Player Inputs, Movement, and Collision

5.1 F009 — Solo Keyboard and Mouse Control Scheme

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. InputController::Poll maps A/D or the arrow keys to horizontal movement, W/Space to jump, J/K/U/H to four combat actions, P to block, F or O to interact, Shift to sprint, and L to dash. In solo mode, the left mouse button can also trigger the basic attack.

Role and interactions. Converting raw input into an action structure prevents GameController from depending directly on individual key codes. It reduces duplication and allows the same player logic to work with several input alternatives.

Source-code/data evidence. AppleKnightAdventure/include/Controller/InputController.h; AppleKnightAdventure/src/Controller/InputController.cpp.

5.2 F010 — Independent Player 1 Controls

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. In local co-op, P1 retains A/D, W/Space, J/K/U/H, P, F, Shift, and L. P1 input is polled separately and dispatched only to Player 1, preventing one key from activating both characters.

Role and interactions. A dedicated key group is essential for predictable co-op on one keyboard. It also supports per-player debugging because the state change can be observed on the matching HUD.

Source-code/data evidence. AppleKnightAdventure/src/Controller/InputController.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

5.3 F011 — Independent Player 2 Controls

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. P2 uses the arrow keys for movement and the numpad for combat: KP1/KP2/KP3/KP0 for the four actions, KP4 for dash, KP5 for block, KP6 for interact, and KP7 for sprint. PollP2 produces a state distinct from solo/P1 input before GameController updates the second character.

Role and interactions. Separating the key groups reduces conflicts on a shared device. Player 2 can independently use portals, interact with objects, and activate skills without requiring Player 1 to confirm the action.

Source-code/data evidence. AppleKnightAdventure/src/Controller/InputController.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

5.4 F012 — Delta-Time Horizontal Movement and Facing

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. Player converts the input axis into a base horizontal speed of approximately 200 pixels per second, updates movement using delta time, and changes facing whenever the axis is nonzero. Animation and hitbox/projectile spawn points read the same facing value.

Role and interactions. Delta-time movement keeps speed approximately stable across different frame rates. A single facing source ensures that attacks, teleportation, projectiles, and the sprite point in the direction intended by the player.

Source-code/data evidence. AppleKnightAdventure/src/Model/Player.cpp; AppleKnightAdventure/include/Utils/Constants.h.

5.5 F013 — Platformer Jumping and Gravity

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. A character receives a jump impulse only while the grounded condition is valid. The initial vertical velocity is approximately -600, after which gravity of 980 pixels per second squared accelerates the character downward. Axis-resolved collision identifies landing and clears falling velocity.

Role and interactions. These values establish a consistent platforming rhythm across the six levels. The same physical foundation is also used by ground enemies and coins thrown from chests.

Source-code/data evidence. AppleKnightAdventure/src/Controller/GameController.cpp; AppleKnightAdventure/include/Utils/Constants.h.

5.6 F014 — Conditional Sprint Speed Increase

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. Holding sprint while horizontal input is present multiplies movement speed by approximately 1.4. Sprint continues through the ordinary collision and animation-state pipeline, so it neither bypasses gravity nor passes through walls.

Role and interactions. Sprint provides a choice between precise positioning and faster traversal. Unlike dash, it grants no invulnerability, has no fixed duration, and requires no cooldown.

Source-code/data evidence. AppleKnightAdventure/src/Controller/GameController.cpp; AppleKnightAdventure/src/Model/Player.cpp.

5.7 F015 — Dash with Cooldown and Invulnerability

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. StartDash sets speed to approximately 520, duration to 0.22 seconds, and cooldown to 1 second. During the dash window, TakeDamage ignores incoming damage; after the timer expires, the character returns to the normal movement state.

Role and interactions. Dash functions as both movement and a timing-based defensive skill. Cooldown prevents continuous use, while world collision still applies and prevents travel through solid tiles.

Source-code/data evidence. AppleKnightAdventure/include/Model/Player.h; AppleKnightAdventure/src/Model/Player.cpp.

5.8 F016 — Block/Parry Damage Reduction

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. While the parry state is active, incoming damage is multiplied by 0.3, reducing it by approximately 70 percent. GameController refreshes a short defensive window while the block control remains held, and each CharacterSkillSet supplies class-specific animation or behavior.

Role and interactions. The mechanic provides defense in addition to dash. The Bulwark Core can upgrade block into complete damage prevention, directly connecting the base combat system to run-build progression.

Source-code/data evidence. AppleKnightAdventure/src/Model/Player.cpp; AppleKnightAdventure/include/Model/CharacterSkillSet.h.

5.9 F017 — Skill Cast Lock and Independent Cooldowns

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. BeginCast/CanStartSkill checks whether the character is alive and whether cooldown, wind-up, and current-cast conditions permit a new skill. Each slot's cooldown timer is updated independently over time.

Role and interactions. Cast locking keeps animations synchronized with hitbox or projectile creation and prevents a charged action from being canceled by key spam. Independent cooldowns create combat rhythm and give the four slots different tactical roles.

Source-code/data evidence. AppleKnightAdventure/src/Model/Player.cpp; AppleKnightAdventure/src/View/SkillBarView.cpp.

5.10 F018 — Axis-Resolved AABB Tile Collision

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. ResolveTileCollisions queries solid tiles on the Main layer, compares the current AABB with each tile, and uses the previous-frame position to determine the penetration axis. X and Y are resolved in order while grounded state and velocity are updated.

Role and interactions. This method produces stable wall, floor, and ceiling behavior on a grid map. Capping delta time at 0.1 seconds reduces the risk that one unusually large physics step tunnels through an obstacle.

Source-code/data evidence. AppleKnightAdventure/src/Controller/GameController.cpp; AppleKnightAdventure/src/Systems/CollisionSystem.cpp.

5.11 F019 — Destructible FakeWall Integrated with Collision

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. An intact FakeWall is included among obstacles alongside solid tiles. After receiving enough valid hits, the object enters its destroyed state, removes its collision, and emits visual and audio feedback.

Role and interactions. The fake wall combines exploration and combat. Before destruction, it blocks players and projectiles like ordinary terrain; afterward, it opens a route without requiring the entire tile map to be replaced.

Source-code/data evidence. AppleKnightAdventure/src/Model/FakeWall.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

5.12 F020 — Out-of-Bounds Death and Checkpoint Respawn

Subsystem: PlayerInputsMovementCollision — **Status:** Integrated into run-time

Definition. When the player's Y position exceeds the map height by approximately two tiles, GameController marks the player dead. If a valid checkpoint exists, the character returns to its respawn position, old projectiles are cleared, and encounters authored beyond the checkpoint can be restored. Co-op also has a rescue branch that respawns a dead player near the surviving teammate.

Role and interactions. The process prevents soft locks after a fall while preserving the level's pacing. A set of kill IDs already credited for score prevents restored enemies from awarding the same score repeatedly.

Source-code/data evidence. AppleKnightAdventure/src/Controller/GameController.cpp; AppleKnightAdventure/src/Model/Checkpoint.cpp.

6 Four 2D Character Skill Sets

6.1 F021 — Knight - Quick Slash

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. Knight's J action creates a fast slash dealing approximately 20 damage, with a 0.35-second cooldown and a 0.20-second active window. The hitbox appears in front of the character according to facing and is deduplicated per target during the attack.

Role and interactions. Quick Slash is a dependable DPS tool for finishing light enemies and opening a combo. Its short cooldown is balanced by lower reach and damage than Heavy Strike or the Ultimate.

Source-code/data evidence. AppleKnightAdventure/src/Model/KnightSkillSet.cpp.

6.2 F022 — Knight - Charged Heavy Strike

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. The K action charges for approximately 0.30 seconds, opens a 0.25-second active window, deals approximately 45 damage, and has a 0.75-second cooldown. Its hitbox is wider and extends farther downward than Quick Slash.

Role and interactions. The wind-up requires the player to read an opponent's telegraph before committing, in exchange for higher damage and a larger hit region. Cast lock keeps the damage instant aligned with the animation.

Source-code/data evidence. AppleKnightAdventure/src/Model/KnightSkillSet.cpp.

6.3 F023 — Knight - Lunge

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. The U action deals approximately 32 damage and propels Knight forward by roughly 97 pixels at approximately 650 pixels per second. Its cooldown is 1.25 seconds, and the movement direction is determined by facing at activation time.

Role and interactions. Lunge closes distance and deals damage, linking traversal and combat. World collision remains active, preventing the attack from passing through walls.

Source-code/data evidence. AppleKnightAdventure/src/Model/KnightSkillSet.cpp.

6.4 F024 — Knight - Bastion-Style 2D Ultimate

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. The H action charges for approximately 0.20 seconds, opens an attack region nearly three times the width of the standard attack, deals approximately 80 damage, and has a 5-second cooldown. Camera and particle feedback emphasize its role as a finishing move.

Role and interactions. The Ultimate handles groups of nearby enemies and creates a burst moment balanced by a long cooldown. Its wide area compensates for Knight's lack of a conventional long-range projectile.

Source-code/data evidence. AppleKnightAdventure/src/Model/KnightSkillSet.cpp; AppleKnightAdventure/src/View/GameView.cpp.

6.5 F025 — Knight - Skill-Set Parry

Subsystem: 2D skills — Status: Integrated into runtime

Definition. KnightSkillSet supplies Knight-specific parry timing and animation while Player applies the shared damage-reduction rule. The behavior preserves the CharacterSkillSet contract used by all four classes.

Role and interactions. Moving parry presentation into the skill set allows a class-specific variant without adding class branches to GameController. It is also direct runtime evidence of polymorphism affecting gameplay.

Source-code/data evidence. AppleKnightAdventure/src/Model/KnightSkillSet.cpp; AppleKnightAdventure/src/Model/Player.cpp.

6.6 F026 — Fighter - Punch

Subsystem: 2D skills — Status: Integrated into runtime

Definition. Fighter's J action deals approximately 20 damage, has a 0.35-second cooldown, and remains active for 0.20 seconds. Its short hitbox favors fast close-range exchanges.

Role and interactions. Punch establishes the brawler play style. The player must remain close to the enemy in exchange for a fast action cycle and steady damage.

Source-code/data evidence. AppleKnightAdventure/src/Model/FighterSkillSet.cpp.

6.7 F027 — Fighter - Combo

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. The K action creates a combo dealing approximately 35 damage, with a 0.75-second cooldown and a 0.35-second active window. The longer active interval supports a multi-beat animation while target deduplication prevents repeated unlimited damage to one enemy.

Role and interactions. Combo is the medium-damage choice between the basic punch and the charged attack. It is most effective after the player has controlled the opponent's position.

Source-code/data evidence. AppleKnightAdventure/src/Model/FighterSkillSet.cpp.

6.8 F028 — Fighter - Charged Energy Punch

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. The U action charges for approximately 0.40 seconds, deals approximately 50 damage, remains active for 0.25 seconds, and has a 1.5-second cooldown. It is Fighter's strongest single melee strike.

Role and interactions. The charge creates positional risk during preparation, while the high damage rewards reading AI behavior and choosing a safe opening. Cast lock preserves animation consistency throughout the action.

Source-code/data evidence. AppleKnightAdventure/src/Model/FighterSkillSet.cpp.

6.9 F029 — Fighter - Energy Orb Ultimate

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. The H Ultimate charges for approximately 0.50 seconds and launches an orb at speed 400. The orb lasts approximately 0.9 seconds, deals approximately 70 damage, and has a 6-second cooldown. Its projectile lifecycle and collision are independent of the player.

Role and interactions. Energy Orb gives Fighter a rare long-range option for finishing an enemy or attacking across a gap. Its long cooldown prevents it from replacing the class's close-combat identity.

Source-code/data evidence. AppleKnightAdventure/src/Model/FighterSkillSet.cpp; AppleKnightAdventure/src/Model/Projectile.cpp.

6.10 F030 — Fighter - Skill-Set Parry

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. FighterSkillSet implements its own block/parry response through the shared CharacterSkillSet contract. Player remains responsible for applying damage reduction and managing life state.

Role and interactions. This division gives Fighter class-appropriate presentation without duplicating the core defensive rule. It makes the composition relationship between Player and its skill strategy explicit.

Source-code/data evidence. AppleKnightAdventure/src/Model/FighterSkillSet.cpp; AppleKnightAdventure/include/Model/CharacterSkillSet.h.

6.11 F031 — Magic Caster - Lightning Thunder

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. The J action charges for approximately 0.40 seconds, finds the nearest visible enemy inside the camera, and deals approximately 45 damage with a 1-second cooldown. A line-of-sight query prevents the lightning from locking onto a target through solid tiles.

Role and interactions. The skill rewards maintaining visibility and camera position without requiring pixel-perfect aiming. Walls remain tactically relevant because they can break target acquisition.

Source-code/data evidence. AppleKnightAdventure/src/Model/MagicCasterSkillSet.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

6.12 F032 — Magic Caster - Fireball

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. The K action charges for approximately 0.35 seconds and launches a Fire projectile at speed 450 with a range of approximately 600. It deals 35 damage, has a 0.8-second cooldown, and applies Fire to the status/reaction system on impact.

Role and interactions. Fireball is a source of Burn and can be either side of an ordered elemental reaction. Its high projectile speed lets Magic Caster maintain pressure from a distance.

Source-code/data evidence. AppleKnightAdventure/src/Model/MagicCasterSkillSet.cpp; AppleKnightAdventure/src/Systems/ElementalSystem.cpp.

6.13 F033 — Magic Caster - Water Wave

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. The U action launches a Water wave at approximately 350 speed, deals 30 damage, has a 0.9-second cooldown, and uses a 0.35-second charge. Water applies Wet, slowing the enemy and preparing a later reaction.

Role and interactions. Its projectile is slower than Fireball but adds control utility. Wet helps maintain distance from melee and flying enemies and makes an elemental sequence safer to execute.

Source-code/data evidence. AppleKnightAdventure/src/Model/MagicCasterSkillSet.cpp; AppleKnightAdventure/src/Systems/ElementalSystem.cpp.

6.14 F034 — Magic Caster - Void Lightning Ultimate

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. The H Ultimate charges for approximately 0.50 seconds, locks onto the nearest enemy with valid line-of-sight, deals approximately 80 damage, and has an 8-second cooldown. The attack applies Void for Corroded or a reaction.

Role and interactions. This is Magic Caster's major single-target burst. Its long cooldown encourages saving it for a boss or priority enemy, while Corroded opens a window in which later attacks deal more damage.

Source-code/data evidence. AppleKnightAdventure/src/Model/MagicCasterSkillSet.cpp; AppleKnightAdventure/src/Systems/ElementalSystem.cpp.

6.15 F035 — Magic Caster - Skill-Set Parry

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. MagicCasterSkillSet implements the same parry interface used by the melee-oriented classes, giving Magic Caster a defensive option when an enemy closes the distance. Player manages the shared damage multiplier and timer.

Role and interactions. The feature prevents Magic Caster from having no response except retreat when pressured at close range. A shared contract lets InputController use the same block action for every class.

Source-code/data evidence. AppleKnightAdventure/src/Model/MagicCasterSkillSet.cpp; AppleKnightAdventure/src/Model/Player.cpp.

6.16 F036 — Ninja - Slash

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. Ninja's J action deals approximately 25 damage, has a 0.4-second cooldown, and remains active for 0.20 seconds. Facing determines the hitbox position, while deduplication limits one target to one hit per slash.

Role and interactions. Its basic damage is slightly higher than that of some classes but still requires close range, supporting a fast play style built around constant repositioning.

Source-code/data evidence. AppleKnightAdventure/src/Model/NinjaSkillSet.cpp.

6.17 F037 — Ninja - Blade Rush

Subsystem: 2D skills — Status: Integrated into runtime

Definition. The K action creates a projectile on the final animation frame. It travels at approximately 480 speed, has a range of 550, deals 40 damage, and has a 1-second cooldown. Triggering creation from an animation event keeps presentation and gameplay synchronized.

Role and interactions. Blade Rush lets Ninja exert pressure at medium range without losing the class's mobile identity. Its high speed makes it useful after teleporting or against a flying enemy.

Source-code/data evidence. AppleKnightAdventure/src/Model/NinjaSkillSet.cpp; AppleKnightAdventure/src/Model/Projectile.cpp.

6.18 F038 — Ninja - Teleport Strike

Subsystem: 2D skills — Status: Integrated into runtime

Definition. The U action finds the nearest enemy within approximately 500 and places Ninja about 80 pixels from the target. If no valid target exists, the skill teleports forward approximately 200 pixels. Its cooldown is approximately 15 seconds.

Role and interactions. The two branches make teleportation both a targeted engagement tool and a traversal or escape option. The long cooldown prevents it from bypassing too many platforms and encounters.

Source-code/data evidence. AppleKnightAdventure/src/Model/NinjaSkillSet.cpp.

6.19 F039 — Ninja - Shadow Clone Ultimate

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. The H Ultimate charges for approximately 0.40 seconds and creates a projectile/clone that travels at speed 420, deals approximately 60 damage, and has a 7-second cooldown. The clone uses the projectile lifecycle and hit-target set.

Role and interactions. Shadow Clone gives Ninja a safe burst option when approaching is not practical. Its clone presentation distinguishes it from Fighter's orb even though both use the shared projectile infrastructure.

Source-code/data evidence. AppleKnightAdventure/src/Model/NinjaSkillSet.cpp; AppleKnightAdventure/src/Model/Projectile.cpp.

6.20 F040 — Ninja - Skill-Set Parry

Subsystem: 2D skills — **Status:** Integrated into runtime

Definition. NinjaSkillSet overrides parry through the same four-class interface. The parry state cannot incorrectly overlap with teleport, cast, or death states.

Role and interactions. The mechanic adds a defensive choice to a speed-oriented build and demonstrates that each class can replace its animation without changing the InputController/GameController contract.

Source-code/data evidence. AppleKnightAdventure/src/Model/NinjaSkillSet.cpp; AppleKnightAdventure/include/Model/CharacterSkillSet.h.

7 Items, Level Completion, and 2D Feedback

7.1 F041 — Coin Pickup and Economy Updates

Subsystem: PowerUpsItems(potion) — **Status:** Integrated into runtime

Definition. When Player touches a Coin, UpdateItems increases the persistent currency balance and item score before removing the object from the world. A coin produced by a chest also receives initial velocity and physical simulation before it can be collected.

Role and interactions. Coins connect gameplay to Shop and long-term saving. Both local players use the same profile economy, so a coin collected by P2 updates the common persistent balance instead of creating two unsynchronized wallets.

Source-code/data evidence. AppleKnightAdventure/src/Controller/GameController.cpp; AppleKnightAdventure/src/Model/Item.cpp.

7.2 F042 — Apple Healing Pickup

Subsystem: PowerUpsItems(potion) — **Status:** Integrated into runtime

Definition. Apple is an immediate healing item that restores approximately 25 HP and clamps the result at maximum health. ItemFactory creates the correct type from level data, while GameController applies the effect on contact.

Role and interactions. Apple provides small sustain during exploration without replacing the stronger Potion or Ghost healing. Floating text makes the restored amount observable.

Source-code/data evidence. AppleKnightAdventure/src/Factories/ItemFactory.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

7.3 F043 — Data-Driven 2D Sprite Animation and State Mapping

Subsystem: 2D Presentation — **Status:** Integrated into runtime

Definition. TextureAtlas parses JSON-defined frames and clips, including per-frame duration, trimming metadata, origin data, looping, and fallback clips. Animator controls playback, direction, speed, seeking, and frame selection, while CharacterRenderer maps the current Player, Enemy, Boss, or Pet state to the appropriate clip and horizontal facing.

Role and interactions. The system converts model state into readable visual feedback without placing animation-file knowledge inside gameplay entities. Player state priority keeps death, hurt, dash, parry, attacks, aerial movement, running, walking, and idle presentation mutually coherent, while the same renderer supports enemy and boss-specific mappings.

Source-code/data evidence. AppleKnightAdventure/src/Model/Player.cpp; AppleKnightAdventure/src/View/CharacterRenderer.cpp; AppleKnightAdventure/src/View/Animator.cpp; AppleKnightAdventure/src/View/TextureAtlas.cpp.

7.4 F044 — Potion: 50 HP Healing, SFX, and Achievement Event

Subsystem: PowerUpsItems(potion) — **Status:** Integrated into runtime

Definition. Potion restores approximately 50 HP, clamps the result at maximum health, plays a dedicated sound event, and sends an event to AchievementManager. One pickup therefore affects the health model, audio, and achievement progression.

Role and interactions. This is the required PowerUpsItems(potion) feature and also demonstrates a gameplay action crossing several systems while retaining a clear owner in GameController.

Source-code/data evidence. AppleKnightAdventure/src/Controller/GameController.cpp; AppleKnightAdventure/src/Systems/AchievementManager.cpp.

7.5 F045 — Adaptive 2D Skill Bar with Live Combat-State Feedback

Subsystem: 2D Interface — Status: Integrated into runtime

Definition. SkillBarView loads four class-specific atlas icons and reads each skill's cooldown, charging, and active state every frame. It renders input labels, numeric cooldown sectors, ready pulses, charging and active rings, Magic Caster element colors, dash readiness, and parry state in responsive solo and two-player layouts.

Role and interactions. The bar is the observable bridge between player input, class-specific skill models, elemental metadata, and combat timing. It lets the player distinguish ready, cooling-down, charging, and active actions without inspecting internal state, and it adapts automatically when the selected class or local-player count changes.

Source-code/data evidence. AppleKnightAdventure/src/View/SkillBarView.cpp; AppleKnightAdventure/src/Controller/GameController.cpp; AppleKnightAdventure/src/Model/Player.cpp.

7.6 F046 — One-Time Chest Opening and Physics-Driven Coin Drops

Subsystem: PowerUpsItems(potion) — Status: Integrated into runtime

Definition. Chest checks that it has not already been opened and then spawns a random number of coin items around its position. Each coin receives initial velocity, gravity, and collision simulation before settling for collection.

Role and interactions. A chest gives a reward more visible presentation than a static pickup. Its opened flag prevents infinite repeated rewards from the same chest state.

Source-code/data evidence. AppleKnightAdventure/src/Model/Chest.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

7.7 F047 — Checkpoint and Encounter Snapshot

Subsystem: 2D level flow — **Status:** Integrated into runtime

Definition. A checkpoint stores the respawn position and the necessary encounter snapshot. On respawn, projectiles are removed and some level-authored enemies beyond the checkpoint are restored, while the set of IDs already credited for score remains intact to prevent score farming.

Role and interactions. The design restores the intended challenge without erasing all progress through the level. It distinguishes state that must reset for an encounter from run data that must remain unchanged.

Source-code/data evidence. `AppleKnightAdventure/src/Model/Checkpoint.cpp`; `AppleKnightAdventure/src/Controller/GameController.cpp`.

7.8 F048 — Color-Paired Portals

Subsystem: World interaction — **Status:** Integrated into runtime

Definition. `TeleportPortal` uses a color/ID property to find its matching endpoint and moves the player to a safe exit point. In local co-op, the partner can be brought through as well, preventing excessive camera separation or leaving one player behind.

Role and interactions. Portals let a level designer connect distant regions of a large map without a continuous physical corridor. Pairing by color supports several independent portal pairs in one level.

Source-code/data evidence. `AppleKnightAdventure/src/Model/TeleportPortal.cpp`; `AppleKnightAdventure/src/Controller/GameController.cpp`.

7.9 F049 — In-Map Signboards and Tutorial Dialogues

Subsystem: Player guidance — **Status:** Integrated into runtime

Definition. Signboard/InMapGuide is an entity with an interaction region and level-authored content. When the player approaches, InteractPrompt appears; confirming the interaction opens a dialogue/tutorial renderer without leaving gameplay.

Role and interactions. Spatially placed instructions let the tutorial teach a control immediately before it is needed. Data-driven content allows a mapper to revise the message without modifying combat code.

Source-code/data evidence. AppleKnightAdventure/src/Model/Signboard.cpp; AppleKnightAdventure/src/View/TutorialRenderer.cpp.

7.10 F050 — LevelCompleteCup Completion Trigger

Subsystem: Campaign level completion — **Status:** Integrated into runtime

Definition. LevelCompleteCup is a goal entity. When the player interacts with it, the cup becomes activated, and GameState::IsLevelComplete recognizes it as a campaign completion condition. The controller then runs the scoring and ResultView pipeline used by the six authored campaign levels.

Role and interactions. The cup supplies a visible finish objective for campaign maps. Its predicate does not itself require every enemy to be dead; level geometry or a gate may enforce the encounter before the player can reach it.

Source-code/data evidence. AppleKnightAdventure/src/Model/LevelCompleteCup.cpp; AppleKnightAdventure/src/Model/GameState.cpp.

8 Completion, Campaign Feedback, and Editor Entry

8.1 F051 — End Checkpoint and Three-State Flag

Subsystem: Campaign level completion — **Status:** Integrated into runtime

Definition. A checkpoint marked as an end-game checkpoint can set the player's completion flag and become an activated completion predicate. Its flag presentation advances through three visible states: uncaptured, flag-out, and a captured loop.

Role and interactions. The visual sequence clearly communicates the state before, during, and after capture. End-checkpoint activation joins the same campaign completion pipeline used for scoring, persistence, ResultView, and sequential level unlocking.

Source-code/data evidence. AppleKnightAdventure/src/Model/GameState.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

8.2 F052 — Performance Score and Star Grading

Subsystem: Level completion and scoring — **Status:** Integrated into runtime

Definition. LevelScoring combines a 40-percent enemy-defeat ratio, a 35-percent item-collection ratio, and a 25-percent time-efficiency ratio. A completed level receives at least one star; composite thresholds of 0.60 and 0.85 award two and three stars respectively.

Role and interactions. The formula rewards combat, exploration, and speed instead of reducing performance to one statistic. Per-level par times allow the same formula to accommodate maps of very different dimensions and encounter density.

Source-code/data evidence. AppleKnightAdventure/src/Model/LevelScoring.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

8.3 F053 — Result Screen: Continue, Retry, and Level Select

Subsystem: Level completion and UI — **Status:** Integrated into runtime

Definition. After time is stopped and the score and stars are calculated, ResultView presents score, elapsed time, stars, new-record markers, and three navigation choices. Continue advances to the next campaign level, Retry reloads the current level, and Level Select returns to the catalog.

Role and interactions. The player can optimise a completed level or continue the campaign without restarting the application. The result is persisted before leaving the view, protecting progress if the application closes immediately afterwards.

Source-code/data evidence. AppleKnightAdventure/src/View/ResultView.cpp;
AppleKnightAdventure/src/Controller/GameController.cpp.

8.4 F054 — Minimap Fog and Exploration Progress

Subsystem: HUD and exploration — **Status:** Integrated into runtime

Definition. MinimapView derives a compact representation from tiles and entities, tracks explored regions, and covers unseen regions with fog. Player position, facing, and important markers update against world and camera state.

Role and interactions. Large maps such as Levels 2–6 become navigable without removing discovery. Exploration state can be persisted through SaveManager so revisiting a campaign level does not arbitrarily forget revealed regions.

Source-code/data evidence. AppleKnightAdventure/src/View/MinimapView.cpp;
AppleKnightAdventure/src/Model/SaveManager.cpp.

8.5 F055 — Achievements and Event Pop-ups

Subsystem: Long-term progression — **Status:** Integrated into runtime

Definition. AchievementManager defines fifteen achievements and receives events from combat, potion use, Shop activity, co-op, and broader progression milestones. A newly satisfied condition updates persistent state and raises a pop-up instead of scanning complete history every frame.

Role and interactions. Achievements connect otherwise separate subsystems into long-term objectives. The dedicated Achievement screen exposes locked and completed states, while event-driven updates keep the gameplay cost bounded.

Source-code/data evidence. AppleKnightAdventure/src/Systems/AchievementManager.cpp; AppleKnightAdventure/src/Model/SaveManager.cpp.

8.6 F056 — Local Score and Time Leaderboards

Subsystem: Long-term progression — **Status:** Integrated into runtime

Definition. SaveManager stores per-level best scores, best times, and a compact local leaderboard of approximately five records per level. Each entry can preserve whether the completion was performed in co-op.

Role and interactions. Score and time create two replay objectives that do not always reward the same route. The local board works without the optional Survival3D network service and therefore remains available in a fully offline session.

Source-code/data evidence. AppleKnightAdventure/src/Model/SaveManager.cpp; AppleKnightAdventure/src/Controller/MenuController.cpp.

8.7 F057 — Music, SFX, Master Volume, and Fullscreen Options

Subsystem: UI and Sounds — **Status:** Integrated into runtime

Definition. OptionsView exposes master, music, and sound-effect volumes as separate controls together with fullscreen selection. Changes are applied to SoundManager or WindowManager and persisted through SaveManager.

Role and interactions. Independent buses let a player retain combat feedback while reducing music, or perform the inverse. Immediate application provides audible and visual feedback before the Options screen is closed.

Source-code/data evidence. AppleKnightAdventure/src/View/OptionsView.cpp; AppleKnightAdventure/src/Systems/SoundManager.cpp; AppleKnightAdventure/src/Systems/WindowManager.cpp.

8.8 F058 — Parallax Background and Menu Blur

Subsystem: Two-dimensional presentation — **Status:** Integrated into runtime

Definition. Background presentation uses several layers with different movement factors; distant layers move less than near layers to create depth in a 2D scene. The Main Menu also applies a two-pass blur shader behind foreground interface panels.

Role and interactions. Parallax communicates camera motion without altering collision, while blur separates interactive foreground content from animated scenery. The shader is stored as a reusable asset instead of being embedded as large source strings in the view.

Source-code/data evidence. AppleKnightAdventure/src/View/MenuView.cpp; AppleKnightAdventure/assets/shaders/blur.fs.

8.9 F059 — Particles, Floating Numbers, Camera Shake, and Hit Stop

Subsystem: Combat feedback — **Status:** Integrated into runtime

Definition. Important impacts can produce a particle burst, floating combat text, a camera-shake request, and a short hit stop. Hit-stop duration is clamped to approximately 0.14 seconds, while attack, element, reaction, wall-break, and victory events select different colours or intensities.

Role and interactions. These four feedback layers make damage readable and increase perceived impact without changing the simulation outcome. Particle pooling limits allocation cost during dense encounters.

Source-code/data evidence. `AppleKnightAdventure/src/Systems/ParticleSystem.cpp`; `AppleKnightAdventure/src/View/FloatingText.cpp`; `AppleKnightAdventure/src/View/GameView.cpp`.

8.10 F060 — MapBuilder Entry, Canvas, and Editor Lifecycle

Subsystem: MapBuilder — **Status:** Integrated into runtime

Definition. `MapBuilderController` owns an editable `GameState`, while `MapBuilderView` presents the canvas, toolbar, minimap, palette, and property panel. Entering and leaving the editor attaches and detaches `GameView` tile pointers deliberately so stale references are not retained.

Role and interactions. `MapBuilder` is a substantial tool inside the same executable, not merely a data file. It lets the team author, save, inspect, and test maps without changing C++ and recompiling for every layout adjustment.

Source-code/data evidence. `AppleKnightAdventure/src/Controller/MapBuilderController.cpp`; `AppleKnightAdventure/src/View/MapBuilderView.cpp`.

9 EnemyBehavior, AI, Build Systems, and MultiplePlayers

9.1 F061 — Melee Enemy Archetype

Subsystem: EnemyBehavior and AI — **Status:** Integrated into runtime

Definition. EnemyFactory constructs a melee archetype with approximately 50 HP and 15 damage. It patrols around its spawn, detects and chases a player within leash rules, enters a telegraphed wind-up, opens a close-range attack, and transitions through hurt and death states when damaged.

Role and interactions. Melee enemies test spacing, parry timing, dash use, and ledge movement. A visible wind-up creates a fair reaction window instead of applying immediate, unannounced contact damage.

Source-code/data evidence. AppleKnightAdventure/src/Factories/EnemyFactory.cpp; AppleKnightAdventure/src/Model/Enemy.cpp.

9.2 F062 — Ranged Enemy and Bomb Projectile

Subsystem: EnemyBehavior and AI — **Status:** Integrated into runtime

Definition. The ranged enemy maintains engagement distance and emits a bomb-like projectile at approximately 250 pixels per second. Its projectile follows an independent life-time and collision path while the enemy retains the shared detection, target, wind-up, hurt, and death state model.

Role and interactions. This archetype forces the player to read lanes and terrain instead of relying exclusively on distance. During co-op, target selection can change to the nearer active player.

Source-code/data evidence. AppleKnightAdventure/src/Model/Enemy.cpp; AppleKnightAdventure/src/Model/Projectile.cpp.

9.3 F063 — Flying Enemy with Two-Axis Movement

Subsystem: EnemyBehavior and AI — **Status:** Integrated into runtime

Definition. A flying enemy is not constrained to platforms: it adjusts both horizontal and vertical motion to approach a target and checks line of sight before the relevant attacks. Detection, leash, hurt, and death still follow the common Enemy state contract.

Role and interactions. Two-axis pursuit adds pressure in vertically layered rooms and gives projectiles and vertical positioning greater value. LOS prevents the flying archetype from attacking directly through solid tiles.

Source-code/data evidence. `AppleKnightAdventure/src/Model/Enemy.cpp`;
`AppleKnightAdventure/src/Controller/GameController.cpp`.

9.4 F064 — Enemy Finite-State Machine and Attack Telegraph

Subsystem: EnemyBehavior and AI — **Status:** Integrated into runtime

Definition. Regular enemies move among Idle, Patrol, Chase, WindUp, Attack, Hurt, and Dead. A ground attack has an approximately 0.75-second wind-up; if its target leaves the valid conditions before release, the attack can be cancelled or return to pursuit.

Role and interactions. Explicit states make transitions observable and keep AI decisions aligned with animation. Telegraphs preserve difficulty while giving the player actionable information.

Source-code/data evidence. `AppleKnightAdventure/include/Model/Enemy.h`;
`AppleKnightAdventure/src/Model/Enemy.cpp`.

9.5 F065 — Line of Sight and Ledge Avoidance

Subsystem: EnemyBehavior and AI — **Status:** Integrated into runtime

Definition. GameController uses tile-aware line-of-sight checks for decisions that require a visible target. A ground enemy also samples supporting tiles ahead; missing ground or a map boundary reverses direction, with a short turn lock of approximately 0.45 seconds.

Role and interactions. The two rules prevent attacks through solid geometry and stop ground enemies from walking meaninglessly into pits. They are deterministic world-sensing rules, not a trained model.

Source-code/data evidence. AppleKnightAdventure/src/Controller/GameController.cpp; AppleKnightAdventure/src/Systems/CollisionSystem.cpp.

9.6 F066 — Three Multi-Phase 2D Bosses

Subsystem: EnemyBehavior and AI — **Status:** Integrated into runtime

Definition. Boss1 has two phases, Boss2 has three, and Boss3 has four; each phase owns a full health bar and a distinct action or damage policy. Boss1 specialises in melee and dash attacks, Boss2 in ranged spread, teleport, healing, and areas of effect, while Boss3 combines melee, sphere, ground-area, and wall-limited beam attacks.

Role and interactions. Nine total phases create three qualitatively different build and execution checks rather than merely increasing enemy HP. Boss HUD and contextual music communicate encounter and phase progression.

Source-code/data evidence. AppleKnightAdventure/src/Model/Boss1.cpp; AppleKnightAdventure/src/Model/Boss2.cpp; AppleKnightAdventure/src/Model/Boss3.cpp.

9.7 F067 — Boss Navigation, Retreat, Gap Traversal, and Unsticking

Subsystem: EnemyBehavior and AI — **Status:** Integrated into runtime

Definition. The Boss base uses flood-fill/BFS navigation over standable cells, checks the boss footprint rather than a single point, and combines this with tile line of sight. Policies can retreat, leap a gap, change elevation, or shove/change direction after stuck detection; co-op targeting selects the nearer viable player.

Role and interactions. Bosses can operate in authored platform spaces instead of moving directly through walls. Footprint-aware standability avoids corridors that fit a player but not the boss body.

Source-code/data evidence. `AppleKnightAdventure/src/Model/Boss.cpp`; `AppleKnightAdventure/include/Model/Boss.h`.

9.8 F068 — Four Rule-Based Pet Behaviours

Subsystem: Companion AI — **Status:** Integrated into single-player 2D

Definition. Fairy seeks nearby items within approximately 400 pixels. Ghost heals outside combat under a finite healing budget. Baby Dragon and Skull select a nearby enemy within roughly 320 pixels, charge, and emit a projectile. Every pet also follows the player using its own movement behaviour.

Role and interactions. The four companions address collection, survivability, or damage without changing the selected player class. Their decisions use explicit distance, combat-state, and cooldown rules rather than machine learning. Pets are intentionally disabled in local co-op.

Source-code/data evidence. `AppleKnightAdventure/src/Model/Pet.cpp`; `AppleKnightAdventure/include/Model/Pet.h`.

9.9 F069 — Twelve Timed Boons and a Three-Card Draft

Subsystem: Power-ups and build construction — **Status:** Integrated into run-time

Definition. BuffSystem implements Vigor, Second Wind, Focus, Adrenaline, Haste, Power, Aegis, Bloodthirst, and four elemental infusions. A boss encounter can pause simulation and present three distinct weighted choices for X/Y/Z input; buff orbs can also spawn periodically under an active-count limit.

Role and interactions. Boons provide short-term healing, regeneration, cooldown reset or acceleration, movement, damage, defence, life-steal, or an elemental conversion. Weighted selection and duplicate prevention preserve meaningful choices inside one draft.

Source-code/data evidence. AppleKnightAdventure/src/Systems/BuffSystem.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

9.10 F070 — Twenty-One Run-Long Cores

Subsystem: Power-ups and build construction — **Status:** Integrated into run-time

Definition. CoreSystem defines thirteen general cores and eight class-restricted cores with rarity, weighted selection, stack caps, and an Epic pity mechanism after an unlucky draft sequence. A three-core draft is awarded after every eight regular kills and after a boss, with boss rewards biased toward higher rarity.

Role and interactions. Cores modify max HP, movement, guarding, cooldown recovery, damage, execution, Burn probability, projectile piercing, Chain Spark, reactions, and class-specific skills. One core provides a single revival at approximately 35 percent HP, illustrating a run-level rule rather than a timed buff.

Source-code/data evidence. AppleKnightAdventure/src/Systems/CoreSystem.cpp; AppleKnightAdventure/include/Systems/CoreSystem.h.

9.11 F071 — Four Elemental Auras and Status Effects

Subsystem: Elemental combat — Status: Integrated into runtime

Definition. Fire, Water, Thunder, and Void map to Burn, Wet, Shocked, and Corroded. Burn deals approximately 7 damage every 0.5 seconds for about 5 seconds; Wet applies a 0.65 speed multiplier; Shocked combines 4-damage ticks with a 0.8 speed multiplier; Corroded increases received damage by approximately 30 percent.

Role and interactions. The four statuses offer damage-over-time, strong slowing, damage plus slowing, and vulnerability. EnemyStatusRenderer exposes icons, timers, and colour treatment so these rules remain observable.

Source-code/data evidence. AppleKnightAdventure/src/Systems/ElementalSystem.cpp; AppleKnightAdventure/src/View/EnemyStatusRenderer.cpp.

9.12 F072 — Twelve Directional Elemental Reactions and Live Codex

Subsystem: Elemental combat — Status: Integrated into runtime

Definition. The reaction matrix defines twelve ordered pairs among four elements, with multipliers approximately between 1.4 and 2.4. Reversing the resident and incoming elements can produce a different reaction. The C key opens a Codex generated from the live reaction data and freezes simulation while the reference is visible.

Role and interactions. The Codex makes a combinatorial combat system inspectable during play. Reading from the same rule source as combat reduces the risk that documentation and implementation diverge.

Source-code/data evidence. AppleKnightAdventure/src/Systems/ElementalSystem.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

9.13 F073 — Local Co-op Dual HUD, Shared Camera, and Shared Upgrades

Subsystem: MultiplePlayers — Status: Integrated as local co-op

Definition. P1 and P2 have separate health, skill cooldowns, and input states, and the HUD renders both sets. A shared camera follows their midpoint and adjusts zoom approximately from 0.62 to 1.0 with separation. Enemies can select the nearer player, while coins, portals, and boon/core grants are coordinated through one shared world.

Role and interactions. The design keeps both players visible without splitting encounter state. Pets are disabled in co-op to control entity complexity and avoid adding an unbalanced third combat helper.

Source-code/data evidence. `AppleKnightAdventure/src/Controller/GameController.cpp`; `AppleKnightAdventure/src/View/HUDView.cpp`; `AppleKnightAdventure/src/View/SkillBarView.cpp`.

9.14 F074 — Timed Random Enemy Spawning in the Campaign

Subsystem: EnemyBehavior and AI — Status: Integrated into runtime

Definition. The dynamic spawner waits a random interval of roughly 22–38 seconds, permits at most three dynamically spawned enemies, and searches approximately 420–1,100 pixels from the player for a location with support and two tiles of free space. Its archetype weights are approximately 70 percent melee and 30 percent ranged.

Role and interactions. The system maintains pressure when a player remains in a large region without spawning directly inside a wall or on top of the player. It is disabled in boss arenas so authored boss encounters are not contaminated by unrelated random enemies.

Source-code/data evidence. `AppleKnightAdventure/src/Controller/GameController.cpp`; `AppleKnightAdventure/src/Factories/EnemyFactory.cpp`.

9.15 F075 — Boss-Arena Portal, Gate Lock, and Player Snapshot/Restore

Subsystem: Two-dimensional boss encounter — **Status:** Integrated into runtime

Definition. Entering a boss encounter captures health, score, skill points, maximum health, damage history, and the Core loadout, establishes the arena portal or gate, and locks the route out. Victory or retry restores those fields according to boss-flow rules instead of allowing a half-completed state to leak into the campaign.

Role and interactions. A dedicated encounter lifecycle makes boss retries reproducible, prevents escape through a portal, and stops rewards or boons from being duplicated. The normal dynamic enemy spawner is also suppressed during the arena state.

Source-code/data evidence. AppleKnightAdventure/src/Controller/GameController.cpp; AppleKnightAdventure/src/Model/TeleportPortal.cpp.

10 Survival3D Features

10.1 F076 — Survival3D Character Select with Two Heroes

Subsystem: 3D Game - **Status:** Integrated

Definition. Survival3D begins in the CharacterSelect phase and lets the player choose either the Knight or the Magic Caster before a run is created. The selection determines the animated model, weapon presentation, base health and movement speed, animation graph, combat kit, and the five actions shown in the HUD.

Role and interactions. The Knight supports close-range combos, guarding, rushing, and a compact high-damage ultimate, while the Magic Caster supports ranged projectiles, area control, and a larger-area ultimate. Each run uses the selected hero's own model, statistics, animation graph, weapon package, and combat loadout.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/View/SurvivalView.cpp.

10.2 F077 — Camera-Relative 3D Movement on the XZ Plane

Subsystem: 3D Game - Movement - Status: Integrated

Definition. Keyboard input is projected onto the camera's forward and right vectors to move the player across the XZ plane. The character faces the intended movement or combat direction. The arena is clamped to approximately ± 19.5 units and contains four solid pillars with a collision radius of 0.95.

Role and interactions. Camera-relative movement keeps WASD intuitive while the camera orbits. Boundary and pillar resolution turn the arena into a constrained tactical space rather than an infinite plane; the same geometry also influences line of sight, enemy movement, root motion, and camera placement.

Source evidence. `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

10.3 F078 — Third-Person Orbit, Zoom, and Camera Collision

Subsystem: 3D Game - Camera - Status: Integrated

Definition. Mouse motion controls third-person orbit, while the mouse wheel changes the requested camera distance over the range 2.15–16.0. Before the final camera position is accepted, the controller resolves walls and pillars and smooths distance, field of view, target, and position.

Role and interactions. Orbit and zoom let the player inspect enemy formations and telegraphs from useful angles. Collision correction prevents the camera from entering pillars or moving behind an arena wall, and target-lock framing can widen the camera to keep both the player and the selected enemy visible.

Source evidence. `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`; `AppleKnightAdventure/src/Survival3D/View/SurvivalView.cpp`.

10.4 F079 — Target Lock and Aim Assistance

Subsystem: 3D Game - Combat - Status: Integrated

Definition. Pressing T or the middle mouse button selects a valid active enemy using distance, facing, line-of-sight, and boss-priority scoring. A lock reticle is rendered over the target, and combat actions can use the locked position or a temporary assisted target rather than relying only on the current model orientation.

Role and interactions. Target lock reduces the control burden when many enemies surround the player, while still permitting free aiming and target changes. The controller safely clears the reference when the target dies or becomes invalid, preventing attacks from following a recycled pool slot.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/View/SurvivalView.cpp.

10.5 F080 — Jumping, Dash Priority, and Invulnerability

Subsystem: 3D Game - Movement - Status: Integrated

Definition. The 3D player uses a jump impulse of approximately 7.25 and gravity of approximately 19.5. Dash or blink is the highest-priority queued combat action, ahead of ultimate, Skill Two, Skill One, and basic attack. The movement is constrained by action state, cooldown, arena bounds, and obstacle collision, and it grants a short invulnerability window.

Role and interactions. Jumping supports vertical presentation and ground-effect avoidance, while dash is the primary response to boss telegraphs and projectile lanes. Priority ordering ensures that simultaneous inputs do not start several incompatible action states in one fixed tick.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/include/Survival3D/Model/SurvivalTypes.h.

10.6 F081 — Knight 3D: Violet Edge

Subsystem: 3D Game - Knight - Status: Integrated

Definition. Violet Edge is the Knight's three-step basic chain. Its three hits deal $24 \times \{1.0, 1.1, 1.6\} \times$ the current damage multiplier. The strike center, radius, facing dot-product, obstacle line of sight, animation event, and combo buffer determine whether each enemy is hit; the third attack uses a slightly wider directional arc.

Role and interactions. Violet Edge is the Knight's repeatable damage source and connects input buffering, the animation state machine, weapon-socket presentation, sword ribbon rendering, hit detection, knockback, sound, and combat feedback. Contact is generated at the authored frame rather than immediately when the button is pressed.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/Animation/AnimationEvents.cpp.

10.7 F082 — Knight 3D: Aegis Counter

Subsystem: 3D Game - Knight - Status: Integrated

Definition. Aegis Counter opens a guard window for 0.65 seconds, extended to 1.0 second by Royal Bulwark. While guarding, incoming damage is multiplied by 0.20 before general damage reduction, and a successfully guarded hit grants 18 ultimate charge.

Role and interactions. The ability rewards reading enemy wind-ups and converts a correctly timed defense into ultimate economy. Despite the word "Counter" in its name, the audited implementation does not automatically emit a separate retaliation-damage hit; the implemented reward is mitigation plus ultimate charge.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/Vfx/SkillVfxPackages.cpp.

10.8 F083 — Knight 3D: Shield Rush

Subsystem: 3D Game - Knight - Status: Integrated

Definition. Shield Rush uses an authored root-motion profile covering approximately 7 units. It sweeps a forward corridor with width $1.1 \times$ the area multiplier, deals base damage $32 \times$ the damage multiplier, applies knockback, and uses an action serial so that one enemy cannot be hit repeatedly by the same rush. Juggernaut widens the corridor by 1.35 and also increases Knight damage.

Role and interactions. The skill closes distance, crosses light formations, and briefly protects the Knight, but collision sweeping prevents it from leaving the arena or passing freely through pillars. It therefore combines mobility, crowd control, target-facing, collision, and action ownership.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/Animation/AnimationGraph.cpp.

10.9 F084 — Knight 3D: Bastion Breaker

Subsystem: 3D Game - Knight - Status: Integrated

Definition. Bastion Breaker is the Knight's ultimate. It consumes 100 ultimate charge, applies base damage $85 \times$ the damage multiplier to valid enemies within radius $5.5 \times$ the area multiplier, checks line of sight, and grants an ultimate invulnerability window. Its impact is synchronized with the authored animation event and a dedicated 0.80-second VFX package.

Role and interactions. The ability provides burst damage against dense waves or a boss and its summons. Gameplay damage remains independent of reduced-motion presentation, while hit stop, camera shake, spatial sound, particles, and impact intensity communicate the contact.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/Vfx/SkillVfxPackages.cpp.

10.10 F085 — Knight 3D: Steel Step

Subsystem: 3D Game - Knight - Status: Integrated

Definition. Steel Step is the Knight-specific dash. Its root-motion profile travels approximately 4 units, its base cooldown is 1.2 seconds before cooldown multipliers, and the action provides a short invulnerability period with Knight-specific trail, sound, and animation presentation.

Role and interactions. Steel Step gives the close-range Knight a reliable way to evade projectiles, cross a telegraphed danger zone, or engage a priority target. Because the same cooldown governs both offensive entry and defensive escape, timing remains a meaningful decision.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/View/SurvivalView.cpp.

10.11 F086 — Magic Caster 3D: Arc Bolt

Subsystem: 3D Game - Magic Caster - Status: Integrated

Definition. Arc Bolt is the Magic Caster's basic projectile. It travels at speed 13, deals base damage $18 \times$ the damage multiplier, uses radius $0.22 \times$ projectile scale, has splash radius $1.2 \times$ area multiplier, lives for approximately 2 seconds, and can pierce according to the current build. Projectiles are acquired from and returned to the fixed projectile pool.

Role and interactions. Arc Bolt supports ranged combat and demonstrates the connection among model/socket placement, free aim, target assistance, projectile simulation, splash and pierce rules, hit feedback, and pooling. Forked Bolt can later mutate the basic attack into three trajectories.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/View/SurvivalView.cpp.

10.12 F087 — Magic Caster 3D: Frost Ring

Subsystem: 3D Game - Magic Caster - Status: Integrated

Definition. Frost Ring applies base damage $22\times$ the damage multiplier to active enemies within radius $4.0\times$ the area multiplier, requires arena line of sight, and applies a 3-second slow. Rifflings and Hex Archers move at 55% speed while slowed, while the Obsidian Brute uses a 65% slow factor.

Role and interactions. The ring is the Magic Caster's immediate anti-swarm and escape tool, compensating for lower base health. It is strongest when melee enemies converge, while distant Rangers may remain outside its radius or behind an occluding pillar.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/Vfx/SkillVfxPackages.cpp.

10.13 F088 — Magic Caster 3D: Gravity Well

Subsystem: 3D Game - Magic Caster - Status: Integrated

Definition. Gravity Well places a field at the clamped aim point, stopping the target before a pillar when necessary. The field lasts 4.0 seconds and uses radius $5.0\times$ the area multiplier. In the audited gameplay logic, Rifflings within 5.5 units have their pursuit direction strongly biased toward the field; the base field does not itself apply periodic damage.

Role and interactions. The field groups light melee enemies for Arc Bolt, Frost Ring, or Astral Tempest. The Event Horizon upgrade additionally destroys hostile projectiles inside the field, turning it into a defensive counter against Ranger and boss volleys.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/Vfx/VfxRuntime.cpp.

10.14 F089 — Magic Caster 3D: Astral Tempest

Subsystem: 3D Game - Magic Caster - Status: Integrated

Definition. Astral Tempest is the Magic Caster's ultimate. It consumes 100 charge, applies base damage $65 \times$ the damage multiplier within radius $8.0 \times$ the area multiplier, checks line of sight, grants invulnerability during the action, and uses a dedicated 1.30-second multi-layer VFX package.

Role and interactions. Its larger radius makes it a wave-clearing and boss-summon tool, while its lower base damage than Bastion Breaker differentiates the two heroes. The gameplay event is separate from visual intensity, so high contrast and reduced motion do not alter balance.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/Vfx/SkillVfxPackages.cpp.

10.15 F090 — Magic Caster 3D: Phase Blink

Subsystem: 3D Game - Magic Caster - Status: Integrated

Definition. Phase Blink is the Magic Caster's short teleport-style dash. The root-motion graph emits the entire approximately 4-unit displacement once when the blink trigger is crossed, the base cooldown is 1.5 seconds, and the controller collision-sweeps or clamps the destination against arena bounds and pillars.

Role and interactions. Phase Blink provides emergency repositioning away from melee pressure, fissures, or projectile lanes. It uses hero-specific animation, sound, and VFX rather than reusing Steel Step presentation, but it cannot teleport outside the arena or through blocked geometry without correction.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/src/Survival3D/View/SurvivalView.cpp.

10.16 F091 — Regular Survival3D Enemy Archetypes

Subsystem: 3D Game - AI - Status: Integrated

Definition. The director can spawn Riftling/Swarm for threat cost 1, Hex Archer/Ranger for cost 3, and Obsidian Brute/Tanker for cost 7. Riftling directly pursues and performs a short melee wind-up; Hex Archer retreats, approaches, or strafes to maintain range before firing; Obsidian Brute advances slowly and performs a large ground slam with knockback.

Role and interactions. Swarm units occupy space, Rangers force lateral movement and line-of-sight management, and Tankers absorb damage while threatening a large local area. The mix in `waves.json` is a selection probability, not a guaranteed final count; threat cost and remaining-budget downgrade rules change the actual composition.

Source evidence. `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`; `AppleKnightAdventure/assets/survival3d/config/waves.json`.

10.17 F092 — Five Multi-Phase 3D Bosses

Subsystem: 3D Game - AI - Status: Integrated

Definition. Waves 10, 20, 30, 40, and 50 use Brood Warden, Hexeye Artillerist, Ironroot Colossus, Eclipse Chimera, and Void Sovereign respectively. Each boss has a distinct movement policy, telegraphed actions, phase or stance transition, and either summons or projectile patterns. Void Sovereign replaces its health bar twice and only dies permanently in phase three.

Role and interactions. The five bosses divide the run into five mechanical chapters: swarm management, projectile-lane management, fissure and slam avoidance, alternating melee/ranged stance response, and a three-bar final examination. They are not enlarged copies of a regular enemy.

Source evidence. `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`; `AppleKnightAdventure/assets/survival3d/config/waves.json`.

10.18 F093 — Data-Driven Wave Director and Difficulty Scaling

Subsystem: 3D Game - 50 Waves - Status: Integrated

Definition. `waves.json` supplies each wave's regular mix, boss identifier, and optional budget multiplier. `balance.json` supplies the base, linear, and quadratic threat-budget terms; spawn interval and minimum interval; active-cap terms; HP and damage scaling; and assist multipliers. The director spends threat by archetype cost and downgrades a selection when the remaining budget cannot pay for it.

Role and interactions. Data-driven rules permit all 50 waves to be tuned without 50 separate functions. Quadratic scaling increases durability and damage smoothly, while mix changes alter the tactical shape of pressure and W3D-48 adds a deliberate $1.15\times$ budget spike.

Source evidence. `AppleKnightAdventure/assets/survival3d/config/waves.json`; `AppleKnightAdventure/assets/survival3d/config/balance.json`.

10.19 F094 — Eighteen Upgrades, Rarity, Stacks, and Pity

Subsystem: 3D Game - Run Build - Status: Integrated

Definition. `Survival3D` defines 18 upgrades and offers three distinct valid choices after a cleared wave. Normal rolls are 62% Common, 28% Rare, 9% Epic, and 1% Legendary. If seven choices pass without an Epic or Legendary offer, the pity rule forces an Epic target. Boss rewards use a stronger table, and character locks and stack caps remove invalid candidates.

Role and interactions. Upgrades can change health, speed, mitigation, cooldowns, area, damage, execution, piercing, shielding, burning, chain hits, revival, risk-reward health, and hero-specific skills. This creates different builds between runs while stack caps prevent a single stat from growing without control.

Source evidence. `AppleKnightAdventure/include/Survival3D/Model/SurvivalTypes.h`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

10.20 F095 — 60 Hz Fixed Timestep and Catch-Up Limit

Subsystem: 3D Game - Runtime - Status: Integrated

Definition. Survival3D clamps the incoming frame delta to 0.1 seconds, accumulates time, and advances simulation in fixed 1/60-second ticks. At most six catch-up ticks are executed in one rendered frame; excess accumulated time is discarded and a dropped-tick counter is incremented.

Role and interactions. Fixed ticks make movement, AI, projectiles, cooldowns, animation events, and wave logic less dependent on rendering FPS. The six-tick cap prevents a long hitch from causing an unbounded catch-up loop and a continuing “spiral of death.”

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp.

10.21 F096 — Fixed Enemy and Projectile Pools

Subsystem: 3D Game - Runtime - Status: Integrated

Definition. Survival3D preallocates 144 enemy slots and 384 projectile slots. Acquire operations pop an index from a free list; release operations reset state, advance generation safety where applicable, and return the index instead of allocating and deleting objects throughout combat. Boss summoning also checks headroom near the pool limit.

Role and interactions. Pooling reduces memory churn and frame-time spikes in dense waves. Its finite capacities form explicit runtime limits that the director, boss policies, diagnostics, and VFX load must respect.

Source evidence. AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp; AppleKnightAdventure/include/Systems/ObjectPool.h.

10.22 F097 — Animation Graph, Frame Events, Combo Buffer, Root Motion, and Runtime IK

Subsystem: 3D Game - Animation - Status: Integrated

Definition. `AnimationGraph` manages deterministic actor states and transitions; `AnimationEvents` dispatches hitbox, projectile, VFX, audio, and footstep events at authored 60 fps frames; the combo buffer controls branch windows; and root-motion profiles produce dash, rush, or blink displacement. Runtime IK computes aim, hand, elbow, foot, normal, and pelvis targets from the current pose and arena floor.

Role and interactions. Combat contact follows animation timing rather than raw button timing. The state machine rejects stale serials, preserves death and boss-phase priority, and root motion remains collision-owned by the controller. IK stabilizes the pelvis and prepares additional presentation goals.

Source evidence. `AppleKnightAdventure/src/Survival3D/Animation/AnimationGraph.cpp`; `AppleKnightAdventure/src/Survival3D/Systems/RuntimeIK.cpp`.

Implementation boundary

Runtime IK calculates feet, aim, hand, elbow, and pelvis presentation targets, but the audited View directly applies the pelvis offset only. A complete visible full-chain or full-body bone solve must not be claimed.

10.23 F098 — 3D Models, Weapon Sockets, and Skill VFX Packages

Subsystem: 3D Game - Presentation - Status: Integrated

Definition. `SurvivalView` incrementally loads the arena, two hero models, eight enemy/boss models, three weapon models, ten skill models, thirteen textures, and the game font. Actor GLBs are checked for required named clips. Weapon attachment searches `weapon_socket.R`, then `weapon.R`, then `hand.R`; dedicated VFX packages map each hero action to layered geometry, trails, glow, particles, impacts, sound metadata, and shake metadata.

Role and interactions. The presentation layer keeps rendering and asset fallback outside AI and combat rules. Correct sockets keep weapons and projectile origins aligned during animation, while procedural fallback geometry keeps the mode visible when a GLB or texture is unavailable.

Source evidence. `AppleKnightAdventure/src/Survival3D/View/SurvivalView.`

`cpp; AppleKnightAdventure/src/Survival3D/Vfx/SkillVfxPackages.cpp.`

10.24 F099 — Accessibility Controls and Performance HUD

Subsystem: 3D Game - UX - Status: Integrated

Definition. F3 toggles the performance HUD, F4 toggles high-contrast presentation, F5 toggles reduced motion, and F6 cycles UI scale through 0.85, 1.00, and 1.20. Accessibility values are persisted in the Survival3D save section. Presentation settings do not change damage, health, enemy scaling, or the wave director.

Role and interactions. High contrast changes palettes, Reduced Motion cuts or shortens notes, particles, ribbons, and shake, UI scale supports different screens, and the performance overlay exposes average and peak frame time, active-object peaks, and dropped ticks.

Source evidence. `AppleKnightAdventure/src/Survival3D/View/SurvivalView.cpp`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

10.25 F100 — Run Results, Coin Rewards, and Local Records

Subsystem: 3D Game - Progression - Status: Integrated

Definition. `RunFailed` and `RunVictory` summarize highest wave, score, survival time, kills, bosses, damage taken, reward, and synchronization state. A locally validated run awards $\min(30, 2 + \lfloor \text{wave}/5 \rfloor + 2 \times \text{bosses})$ coins. Claimed run IDs prevent the same result from awarding currency twice, and local records are sorted into a top-five display.

Role and interactions. The mode retains progression value when the remote service is disabled: `SaveManager` records the run and economy locally, while an optional pending submission can synchronize later without replacing the local source of continuity.

Source evidence. `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`; `AppleKnightAdventure/src/Model/SaveManager.cpp`.

11 Sound, Persistence, OOD, and Design Patterns

11.1 F101 — Centralized Audio Manifest

Subsystem: Sound - Status: Integrated

Definition. `audio_manifest.json` declares 83 audio samples, 75 logical sound events, and 4 music tracks. `SoundManager` parses this manifest so gameplay requests a logical event name instead of hard-coding a WAV path at every call site.

Role and interactions. This indirection allows samples, volume, pitch range, cooldown, and layered composition to be changed without editing combat rules. The manifest also provides a centralized, auditable catalog for sounds used by 2D gameplay, menus, and Survival3D.

Source evidence. `AppleKnightAdventure/assets/sounds/audio_manifest.json`;
`AppleKnightAdventure/src/Systems/SoundManager.cpp`.

11.2 F102 — Non-Repeating Random Samples and Pitch Variation

Subsystem: Sound - Status: Integrated

Definition. When a logical event contains multiple samples, `SoundManager` chooses among them randomly while avoiding the immediately previous selection when alternatives exist. Playback pitch is randomized within the manifest-defined range, and per-event state remembers the most recent sample.

Role and interactions. The mechanism reduces mechanical repetition in frequently repeated attacks, footsteps, impacts, and enemy reactions without requiring a separate logical event for every variation.

Source evidence. `AppleKnightAdventure/src/Systems/SoundManager.cpp`;
`AppleKnightAdventure/assets/sounds/audio_manifest.json`.

11.3 F103 — Per-Event Cooldowns and Layered Playback

Subsystem: Sound - Status: Integrated

Definition. A logical sound event can define a cooldown that suppresses excessive retriggering, and an event can request several audio layers in one gameplay call. `SoundManager` checks the last playback time before creating new voices and then resolves the configured sample layers.

Role and interactions. Cooldowns protect mix clarity and runtime cost when many hits land in one frame, while layering creates richer boss and ultimate sounds without coupling gameplay code to several sample files.

Source evidence. `AppleKnightAdventure/src/Systems/SoundManager.cpp`; `AppleKnightAdventure/assets/sounds/audio_manifest.json`.

11.4 F104 — Spatial Sound Attenuation and Stereo Panning

Subsystem: Sound - Status: Integrated

Definition. `PlaySoundAt` receives a world position, listener position, and audible range, attenuates volume with distance, and computes left-right stereo panning. `Survival3D` uses positional playback for hero abilities and other world-space combat cues.

Role and interactions. Spatial sound communicates threats beyond the camera center and prevents every event in the arena from playing at the same loudness. It complements visual telegraphs, target lock, and the third-person camera rather than replacing them.

Source evidence. `AppleKnightAdventure/src/Systems/SoundManager.cpp`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

11.5 F105 — Context-Sensitive Background Music

Subsystem: Sound - Status: Integrated

Definition. Controllers select logical music tracks for menu, tutorial, ordinary gameplay, and boss contexts. In Survival3D, regular waves use `bgm_gameplay` and boss waves switch to `bgm_boss`; `SoundManager` owns loading and volume rather than requiring every View to load music files.

Role and interactions. Music signals transitions between preparation, exploration, ordinary combat, and a milestone boss. Logical tracks remain compatible with the options menu's music-volume setting and centralized manifest.

Source evidence. `AppleKnightAdventure/src/Systems/SoundManager.cpp`;
`AppleKnightAdventure/src/Controller/GameController.cpp`.

11.6 F106 — Save Schema Version 2 with Temporary File and Backup

Subsystem: Persistence - Status: Integrated

Definition. `SaveManager` writes schema version 2 through a temporary `.tmp` file and maintains a `.bak` copy before replacing the primary save. The save contains campaign progression, currency, selection, records, achievements, minimap state, options, and Survival3D data.

Role and interactions. The temporary-write and backup sequence reduces the chance that an interrupted write leaves only a partially serialized primary file. It provides a practical recovery path rather than merely catching a JSON exception after data has been lost.

Source evidence. `AppleKnightAdventure/src/Model/SaveManager.cpp`;
`AppleKnightAdventure/include/Model/SaveManager.h`.

11.7 F107 — Validated Save Loading, Clamping, Deduplication, and Fallback

Subsystem: Persistence - Status: Integrated

Definition. Save loading supplies defaults for missing fields, clamps out-of-range values, limits Survival3D collections to 20 runs, 50 pending submissions, and 100 claimed reward IDs, and deduplicates identifiers where required. If the primary file cannot be used, the backup path can be attempted.

Role and interactions. These defenses prevent an old, incomplete, or manually edited JSON save from crashing the game or creating unbounded in-memory collections. They are local integrity checks, not encryption, authentication, or complete anti-cheat protection.

Source evidence. `AppleKnightAdventure/src/Model/SaveManager.cpp`.

11.8 F108 — Asynchronous Asset Preparation with Main-Thread GPU Upload

Subsystem: Runtime Architecture - Status: Integrated

Definition. `AssetManager` uses worker queues for CPU-side file reading and decoding, then transfers texture-creation work to the main thread. Loading progress and errors are aggregated into loading state instead of allowing a worker to call graphics-context operations directly.

Role and interactions. This producer-consumer split improves startup responsiveness and respects ownership of the raylib/OpenGL context. Survival3D then performs its own incremental render-thread model and texture loading after the 2D atlas phase completes.

Source evidence. `AppleKnightAdventure/src/View/AssetManager.cpp`; `AppleKnightAdventure/include/View/AssetManager.h`.

11.9 F109 — Layered Renderer, Stable Z-Sort, and MPSC Submission Queue

Subsystem: Runtime Architecture - Status: Integrated

Definition. The renderer accepts commands from multiple producers through a multi-producer, single-consumer queue, assigns render layers, and performs a stable sort by Z before drawing. Views submit render data rather than independently deciding the entire global order.

Role and interactions. Stable ordering prevents sprites or UI elements at equal depth from flickering unpredictably. Queue-based submission reduces coupling between producers and the render frame and supports backgrounds, tiles, entities, particles, HUD elements, and modal interfaces.

Source evidence. `AppleKnightAdventure/src/View/Renderer.cpp`; `AppleKnightAdventure/include/View/RenderTypes.h`.

11.10 F110 — Model-View-Controller Separation

Subsystem: OOD and Design Patterns - Status: Integrated

Definition. Model directories contain entities and persistent or gameplay state, View directories contain rendering and interface presentation, and Controllers coordinate input, rules, and state transitions. For example, `Player` does not draw itself, `PrepareView` does not construct the complete game state, and `SurvivalController` is separate from `SurvivalView`.

Role and interactions. The separation allows presentation or data representation to change with less cross-system impact and makes subsystem auditing clearer. The architecture is accurately described as MVC-like; not every dependency is perfectly isolated and the report does not claim strict textbook MVC compliance.

Source evidence. `AppleKnightAdventure/include/Model`; `AppleKnightAdventure/include/View`; `AppleKnightAdventure/include/Controller`.

11.11 F111 — Inheritance, Composition, and Runtime Polymorphism

Subsystem: Object-Oriented Design - Status: Integrated

Definition. `Entity` is an abstract base; `Character` derives from `Entity`; and `Player`, `Enemy`, `Boss`, and `Pet` derive through the character hierarchy, with concrete boss classes deriving from `Boss`. `Player` composes a `unique_ptr<CharacterSkillSet>`, active timed buffs, and a Core loadout, while shared resources use `shared_ptr` when ownership is intentionally shared.

Role and interactions. Inheritance centralizes lifecycle and behavior contracts, composition allows a skill set to change at runtime, and smart pointers communicate ownership. The large `GameController` and some `dynamic_cast` usage remain trade-offs, so the project must not be labeled perfectly SOLID.

Source evidence. `AppleKnightAdventure/include/Model/Entity.h`; `AppleKnightAdventure/include/Model/Character.h`; `AppleKnightAdventure/include/Model/Player.h`.

11.12 F112 — Singleton Services and Shared Views

Subsystem: Design Patterns - Status: Integrated

Definition. `GameController`, `SaveManager`, `SoundManager`, `Renderer`, `Survival3D` services, and several Views expose `GetInstance()`, hide construction, and disallow copying. One instance owns application-wide resources or state.

Role and interactions. Singleton access reduces the need to pass common services through every call site in a small game, but it increases global coupling and makes isolated tests more difficult. Both the convenience and the trade-off are part of the design-pattern evaluation.

Source evidence. `AppleKnightAdventure/src/Controller/GameController.cpp`; `AppleKnightAdventure/src/Model/SaveManager.cpp`; `AppleKnightAdventure/src/Systems/SoundManager.cpp`.

11.13 F113 — Enemy, Item, and Level Factories

Subsystem: Design Patterns - Status: Integrated

Definition. `EnemyFactory` and `ItemFactory` map enum or loaded data to concrete objects, while `LevelFactory` selects a source and constructs a common game-state representation. Callers request what must be created without depending on every subclass constructor.

Role and interactions. Factories centralize data-to-object mapping, allowing loaders and spawners to reuse construction rules. Adding a supported archetype generally changes factory and data mappings rather than every gameplay call site.

Source evidence. `AppleKnightAdventure/src/Factories/EnemyFactory.cpp`; `AppleKnightAdventure/src/Factories/ItemFactory.cpp`; `AppleKnightAdventure/src/Factories/LevelFactory.cpp`.

11.14 F114 — Adapters for .lvl and .ldtk Sources

Subsystem: Design Patterns - Status: Integrated

Definition. `ILevelSourceAdapter` defines a common level-reading contract. `LegacyLevelAdapter` handles the custom .lvl format, while `LDtkLevelAdapter` handles .ldtk project data. `LevelFactory` selects the adapter by source type or extension and receives one shared model.

Role and interactions. The Adapter pattern keeps newer campaign content and older custom maps playable without teaching `GameController` two parsing grammars. This is an implemented data boundary rather than a pattern shown only in a class diagram.

Source evidence. `AppleKnightAdventure/src/Factories/LevelSourceAdapter.cpp`; `AppleKnightAdventure/src/Factories/LevelFactory.cpp`.

11.15 F115 — Command, Composite, Undo, and Redo

Subsystem: Design Patterns and MapBuilder - Status: Integrated

Definition. MapBuilder wraps edits in Command objects and CommandManager stores undo and redo history. A paste or bucket fill that changes many cells can group child commands into a CompositeCommand, allowing the entire user operation to reverse as one unit.

Role and interactions. The pattern decouples editor input from mutation and provides recoverable editing history. Composite ownership prevents a large paste from requiring hundreds of separate Undo presses.

Source evidence. AppleKnightAdventure/src/Model/Command.cpp; AppleKnightAdventure/src/Controller/MapBuilderController.cpp.

11.16 F116 — Object Pooling for Effects and Combat Entities

Subsystem: Design Patterns - Status: Integrated

Definition. The 2D particle system uses a generic ObjectPool, while Survival3D maintains specialized fixed enemy and projectile pools. Inactive objects are reset and reused rather than allocated and destroyed for every particle, hit, projectile, or spawn.

Role and interactions. Pooling reduces allocator pressure, fragmentation, and hitching in effect-heavy scenes and dense Survival3D waves. Capacity is an observable resource boundary, not evidence of infinite entities.

Source evidence. AppleKnightAdventure/include/Systems/ObjectPool.h; AppleKnightAdventure/src/Systems/ParticleSystem.cpp; AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp.

11.17 F117 — Quadtree Broad-Phase Collision Queries

Subsystem: Design Patterns and Collision - Status: Integrated

Definition. `CollisionSystem` rebuilds a Quadtree from active entities and exposes rectangle queries for nearby candidates. Melee and combat narrow-phase checks process the returned subset instead of comparing every entity against every hitbox.

Role and interactions. Spatial partitioning reduces potential pair checks in enemy- and item-heavy levels, especially the larger later campaign stages. A Quadtree result remains a broad-phase candidate list; AABB or attack-specific narrow checks still decide actual contact.

Source evidence. `AppleKnightAdventure/src/Systems/Quadtree.cpp`; `AppleKnightAdventure/src/Systems/CollisionSystem.cpp`; `AppleKnightAdventure/src/Controller/GameController.cpp`.

11.18 F118 — Animation-Event Callback Dispatch

Subsystem: Design Patterns and Animation - Status: Integrated

Definition. Animation event tracks dispatch callbacks for hitboxes, projectiles, sound, VFX, footsteps, and phase events at defined frames. The animation system emits an event occurrence, while the owning controller still validates the actor, state, action serial, and target before applying gameplay effects.

Role and interactions. This Observer/callback-like boundary allows authored clip timing to move without scattering independent magic delays across update loops. It is essential to readable 3D combos, projectile release, and boss telegraphs.

Source evidence. `AppleKnightAdventure/src/Survival3D/Animation/AnimationEvents.cpp`; `AppleKnightAdventure/src/Survival3D/Animation/AnimationGraph.cpp`.

11.19 F119 — Offline Submission Queue, Background Worker, and Exponential Retry

Subsystem: Survival3D Service - Status: Integrated

Definition. `SurvivalRunService` stores pending results locally, uses a worker thread for HTTP work, and retries retryable failures with exponential backoff derived from $2000 \ll \min(7, \text{retryCount})$ milliseconds and capped at 300000 milliseconds. `services.json` currently sets `enabled=false` and points to localhost.

Role and interactions. The design is local-first: a run is validated, saved, rewarded, and queued without blocking rendering. When enabled, the worker obtains guest identity, submits results with an idempotency key, and refreshes the leaderboard.

Source evidence. `AppleKnightAdventure/src/Survival3D/Systems/SurvivalRunService.cpp`; `AppleKnightAdventure/assets/survival3d/config/services.json`.

11.20 F120 — Server Validation, Idempotency, and Validated Leaderboard

Subsystem: Survival3D Backend - Status: Integrated

Definition. `AegisRiftServer` limits request size, validates character, configuration, wave 1–50, score, time, kills, boss count, damage, and victory consistency, then recomputes ranked score as $10 \times \text{kills} + 10000 \times \text{bosses} + 500 \times \text{highestWave}$. Player plus idempotency key prevents duplicate completion, and only validated runs enter the best-run-per-player leaderboard.

Role and interactions. The backend does not trust the submitted client score, prevents repeated completion records, and persists JSON under a mutex through temporary and backup files. However, guest X-Player-Id over a localhost HTTP demo is not production authentication or transport security.

Source evidence. `AppleKnightAdventure/backend/survival3d/SurvivalServerCore.cpp`; `AppleKnightAdventure/backend/survival3d/main.cpp`.

Backend and scoring boundary

This is a local/demo backend, not a production-secure service. The client display score awards normal kills by current wave and bosses by milestone, while the server recomputes ranked score with the formula above; the two scoring definitions are not currently identical and should be reconciled.

12 Shop, Custom Levels, MapBuilder, UI, and Build

12.1 F121 — Shop Catalog, Purchase, Unlock, and Equip Flow

Subsystem: Shop and Economy - Status: Integrated into Runtime

Definition. The Shop presents Characters and Pets tabs, animated previews, HP/ATK/Speed information, descriptions, prices, current coins, lock state, and Buy or Equip actions. `ShopController` checks the persistent unlock list and balance, deducts currency, records the normalized character or pet unlock identifier, equips the purchase, triggers feedback and the purchase achievement, and saves the updated profile.

Role and interactions. The feature closes the economy loop from campaign rewards to persistent customization. Knight is available by default; priced characters and pets remain locked until purchased, while previously owned entries take the equip path without charging the player again.

Source evidence. `AppleKnightAdventure/src/Controller/ShopController.cpp`; `AppleKnightAdventure/src/View/ShopView.cpp`; `AppleKnightAdventure/src/Model/SaveManager.cpp`.

12.2 F122 — Custom Level Browser and User-Map Loading

Subsystem: Custom Content - Status: Integrated

Definition. The menu provides access to custom-level selection, passes the chosen source to `LevelFactory` and its adapter, and constructs a normal `GameState`. Invalid files enter a controlled error state instead of passing a null or incomplete level into gameplay.

Role and interactions. The browser turns `MapBuilder` output into replayable content and keeps the legacy custom format usable beside `LDtk` campaign data. A custom completion does not automatically unlock the six authored campaign levels.

Source evidence. `AppleKnightAdventure/src/Controller/MenuController.cpp`; `AppleKnightAdventure/src/Factories/LevelFactory.cpp`.

12.3 F123 — MapBuilder Brush, Eraser, Bucket Fill, and Selection

Subsystem: MapBuilder - Status: Integrated

Definition. The editor includes Brush placement, Eraser removal, contiguous Bucket Fill, and Select or box-select operations for grouped editing. Camera, pan, and zoom tools are separated from mutation tools so navigation clicks do not unintentionally paint the map.

Role and interactions. These tools cover workflows from single-cell detail to large-region construction. Bucket Fill is represented through command history so one Undo can reverse the complete fill operation.

Source evidence. `AppleKnightAdventure/src/Controller/MapBuilderController.cpp`; `AppleKnightAdventure/src/View/MapBuilderView.cpp`.

12.4 F124 — MapBuilder Layers, Tilesets, Entity Palette, and Properties

Subsystem: MapBuilder - Status: Integrated

Definition. MapBuilder edits Background, Main, and Foreground layers, supports layer locking, provides multiple tilesets and an entity palette, and exposes editable properties. Main-layer cells can be marked solid so gameplay collision reads the authored geometry.

Role and interactions. The three layers separate rear decoration, gameplay and collision, and front decoration. The palette supplies data-defined spawn points, enemies, items, portals, checkpoints, bosses, and tutorial markers.

Source evidence. `AppleKnightAdventure/src/View/MapBuilderView.cpp`; `AppleKnightAdventure/src/Controller/MapBuilderController.cpp`.

12.5 F125 — MapBuilder Copy, Paste, and Edit History

Subsystem: MapBuilder - Status: Integrated

Definition. A selection containing tiles and entities can be copied and pasted at a new location. Paste constructs a `CompositeCommand` from its child edits; Z requests Undo and Y requests Redo through `CommandManager`.

Role and interactions. Copy and paste accelerate repeated structures while retaining recoverable editing. Command ownership protects temporary entities and avoids dangling references as the composite operation is undone and redone.

Source evidence. `AppleKnightAdventure/src/Controller/MapBuilderController.cpp`; `AppleKnightAdventure/src/Model/Command.cpp`.

12.6 F126 — MapBuilder Safe Save/Load and LDtk Import

Subsystem: MapBuilder - Status: Integrated

Definition. SaveMap sanitizes the requested file name and writes the named map plus a playable mirror when required. Load uses the native file dialog for .lvl or .ldtk sources and passes import through LevelFactory. Existing tile references are detached before replacement data is attached.

Role and interactions. Name filtering reduces unintended file paths, LDtk import permits continued editing of authored content, and adapters keep parsing details outside the editor controller. Detaching old references protects ownership during a complete map replacement.

Source evidence. AppleKnightAdventure/src/Controller/MapBuilderController.cpp; AppleKnightAdventure/src/Platform/FileDialog.cpp; AppleKnightAdventure/src/Factories/LevelSourceAdapter.cpp.

12.7 F127 — Temporary MapBuilder Playtest and Editor Return

Subsystem: MapBuilder - Status: Integrated

Definition. Playtest serializes the current editor state to a temporary map with ID -99, enters gameplay, and retains a flag that returns the user to the editor after completion or exit. The temporary run is not treated as a genuine campaign completion.

Role and interactions. The edit-test loop shortens validation of collision, spawn points, portals, enemies, and level completion. A distinct temporary identity prevents an unnamed playtest from overwriting a normal user map or advancing campaign progression.

Source evidence. AppleKnightAdventure/src/Controller/MapBuilderController.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

12.8 F128 — Application UI States and Modal-Layer Coordination

Subsystem: User Interface - Status: Integrated into Runtime

Definition. MenuController, MenuView, and UIManager coordinate the Main Menu, Pause overlay, Error dialog, Shop, Level Select, Custom Maps, Leaderboard, Achievements, Options, gameplay HUD and Skill Bar, and Result screen. The layer stack distinguishes persistent gameplay presentation from modal screens, dims lower content, and centralizes transition and back behavior.

Role and interactions. Centralized UI state prevents incompatible screens from consuming input or rendering as the active modal simultaneously. It also keeps Pause and Result behavior consistent with controller state while allowing HUD and Skill Bar presentation to remain separately managed.

Source evidence. AppleKnightAdventure/include/View/UIStateManager.h; AppleKnightAdventure/src/View/UIStateManager.cpp; AppleKnightAdventure/src/View/MenuView.cpp; AppleKnightAdventure/src/Controller/MenuController.cpp; AppleKnightAdventure/src/Controller/GameController.cpp.

12.9 F129 — CMake C++17 Build, Asset Synchronization, and Separate Server Target

Subsystem: Build and Deployment - Status: Integrated

Definition. CMake configures C++17, raylib/OpenGL, nlohmann_json 3.11.3, and required Windows libraries. A tracked asset-sync target copies runtime assets into the build directory. On Windows, AegisRiftServer is built as a separate executable from the main game.

Role and interactions. The build declaration makes dependencies explicit and keeps runtime assets synchronized even when only data changes. Separating the server executable allows the game to operate in local-only mode without launching the backend service.

Source evidence. AppleKnightAdventure/CMakeLists.txt; AppleKnightAdventure/backend/survival3d/main.cpp.

12.10 F130 — Data-Driven Configuration, Fallbacks, and Diagnostic Boundaries

Subsystem: Configuration and Reliability - Status: Integrated with Explicit Fallbacks

Definition. Wave rules, balance coefficients, service settings, and audio definitions are stored in JSON. Loaders provide defaults, validation, and fallback so missing secondary data or a disabled remote endpoint does not prevent the main game from running. Performance diagnostics, error views, and logs expose configuration and runtime failures.

Role and interactions. Designers can tune data without recompiling, while fallback behavior prevents one optional file from crashing the entire application. The same boundary also requires honest reporting of settings that are declared but not yet fully consumed.

Source evidence. `AppleKnightAdventure/assets/survival3d/config/waves.json`; `AppleKnightAdventure/assets/survival3d/config/balance.json`; `AppleKnightAdventure/assets/survival3d/config/services.json`; `AppleKnightAdventure/assets/sounds/audio_manifest.json`.

Configuration boundaries

`services.json` declares `connectTimeoutMs` and `requestTimeoutMs`, but the current WinHTTP client hard-codes 3000/3000/3000/8000 ms and reads only `enabled` and `baseUrl`. Client display scoring and server ranked scoring also use different formulas. Both differences should be corrected before claiming complete service-config or scoring parity.

13 Six 2D Campaign Levels — A Separate Catalog

This chapter describes L2D-01–L2D-06 as authored campaign content. None of these six entries is used to increase the count of 130 independent features. The runtime loads index 0 from each campaign LDtk project; secondary levels named *Dungeon* inside several project files do not belong to the current campaign path.

Table 3: Summary of the six runtime 2D levels

ID	Level and source	Dimensions	Enemies and bosses	Authored items	Par (s)
L2D-01	Tutorial, tutorial.ldtk	$7,360 \times 320$ px; 460 $\times 20$ cells	4 melee enemies	No scored item; TotalItems = 0	300
L2D-02	Forest, lv12.ldtk, index 0	$3,280 \times 3,472$ px; 205 $\times 217$ cells	53 regular enemies and Boss1: 54 total	6 chests, 18 coins, 17 potions: target 41	240
L2D-03	Forest, lv13.ldtk, index 0	$4,128 \times 1,488$ px; 258 $\times 93$ cells	172 regular enemies and Boss1: 173 total	12 chests, 17 coins, 30 potions: target 59	270
L2D-04	Forest, lv14.ldtk, index 0	$1,456 \times 1,376$ px; 91 $\times 86$ cells	234 flying enemies and Boss2: 235 total	48 potions; no authored chest or coin	300
L2D-05	Forest, lv15.ldtk, index 0	$2,720 \times 2,000$ px; 170 $\times 125$ cells	298 regular enemies and Boss2: 299 total	12 chests, 31 potions: target 43	330
L2D-06	ColdCorridor, lv16.ldtk, Level_0	$2,720 \times 2,000$ px; 170 $\times 125$ cells	84 regular enemies plus Boss1, Boss2, and Boss3: 87 total	16 chests, 91 coin entities worth 240, 18 potions: target 125	360

13.1 L2D-01 — Tutorial

Runtime source and principal parameters

Source: AppleKnightAdventure/assets/levels/tutorial.ldtk. **Identifier:** Tutorial. **Theme:** Forest. **Size:** $7,360 \times 320$ pixels on a 16-pixel grid, or 460×20 cells. **Par time:** 300 seconds.

The level contains four melee enemies. Three training enemies use approximately 50 HP and 10 damage, while the final, stronger target uses approximately 150 HP and 20 damage. No item contributes to the level-scoring item denominator because `TotalItems` is zero. The authored map contains 45 entities: one solo spawn, six Signboards, twenty-two InMapGuide instances, ten portals, one checkpoint statue, four enemies, and one LevelCompleteCup. It also contains 1,604 authored tiles and 1,372 non-zero collision cells.

The route is deliberately linear. The first area introduces movement and directional guides. The second demonstrates interaction, checkpoint activation, and portals. The third isolates dash movement. The fourth asks the player to use the U, J, and K actions against three training enemies. The fifth teaches the ultimate against the stronger enemy before the final trophy. Five colour-linked portal pairs connect cell positions 33–85, 118–170, 198–250, 288–340, and 373–425.

13.2 L2D-02 — Forest, Level 2

Runtime source and principal parameters

Source: AppleKnightAdventure/assets/levels/lv12.ldtk, index 0. **Theme:** Forest. **Size:** $3,280 \times 3,472$ pixels, or 205×217 grid cells. **Par time:** 240 seconds.

The runtime level contains 53 regular enemies—24 melee, 9 ranged, and 20 flying—plus Boss1, producing 54 enemy or boss instances. The authored item denominator is 41: six chests, eighteen coins, and seventeen potions. World interactions include six intermediate checkpoints, one LevelCompleteCup, a pair of colour-linked portals, and one FakeWall.

The tall, multi-storey layout makes vertical platforming and the minimap important. All three regular enemy archetypes are introduced in a meaningful mix. Chests and potions establish a recovery rhythm before Boss1's two full HP phases. The portal pair and FakeWall provide two distinct examples of authored world interaction rather than pure combat.

13.3 L2D-03 — Forest, Level 3

Runtime source and principal parameters

Source: AppleKnightAdventure/assets/levels/lv13.ldtk, index 0. **Theme:** Forest. **Size:** $4,128 \times 1,488$ pixels, or 258×93 grid cells. **Par time:** 270 seconds.

Level 3 contains 172 regular enemies—69 melee, 44 ranged, and 59 flying—and one Boss1, for 173 enemy or boss instances. Its 59 authored scoring items consist of twelve chests, seventeen coins, and thirty potions. The level also contains twelve intermediate checkpoints, one LevelCompleteCup, six portals forming three colour pairs, and one FakeWall.

Compared with Level 2, this map is wider and substantially denser. Three portal pairs divide the long route into recognisable sections, while the unusually high numbers of checkpoints, chests, and potions support the extended journey. Boss1 then verifies that the player can carry a developed boon/core build into a familiar two-phase boss under greater preceding attrition.

13.4 L2D-04 — Forest, Level 4

Runtime source and principal parameters

Source: AppleKnightAdventure/assets/levels/lvl4.ldtk, index 0. **Theme:** Forest. **Size:** $1,456 \times 1,376$ pixels, or 91×86 grid cells. **Par time:** 300 seconds.

This specialised level contains 234 flying enemies and Boss2, for a total of 235 enemy or boss instances. Its authored items are forty-eight potions; no chest or coin is placed directly in the level. The map contains twelve checkpoints, one completion cup, two portals, and sixteen FakeWalls.

Although the physical map is comparatively compact, its density turns it into a focused flying-enemy challenge. The sixteen FakeWalls change traversal paths and sight lines, while the large potion supply sustains the player before Boss2's three ranged-oriented phases, which combine projectile spreads, teleportation, self-healing, and telegraphed areas of effect.

13.5 L2D-05 — Forest, Level 5

Runtime source and principal parameters

Source: AppleKnightAdventure/assets/levels/lvl5.ldtk, index 0. **Theme:** Forest. **Size:** $2,720 \times 2,000$ pixels, or 170×125 grid cells. **Par time:** 330 seconds.

Level 5 contains the campaign's largest regular-enemy count: 298 regular enemies—86 melee, 30 ranged, and 182 flying—plus Boss2, producing 299 enemy or boss instances. The 43 authored scoring items consist of twelve chests and thirty-one potions; there is no directly authored coin entity. Seventeen checkpoints, one completion cup, four portals forming two pairs, and twelve FakeWalls organise the map.

The level is strongly weighted toward flying enemies while preserving enough melee and ranged units to create three-role encounters. Numerous checkpoints and chests divide the large map into recoverable stages. The two portal pairs and FakeWalls provide route changes and shortcuts, and the second Boss2 appearance tests consistency after the campaign's highest regular-enemy attrition.

13.6 L2D-06 — ColdCorridor, Level 6

Runtime source and principal parameters

Source: `AppleKnightAdventure/assets/levels/lvl6.ldtk`, identifier `Level_0`. **Theme:** `ColdCorridor`. **Size:** $2,720 \times 2,000$ pixels, or 170×125 grid cells. **Par time:** 360 seconds.

The campaign finale contains 84 regular enemies—33 melee, 17 ranged, and 34 flying—plus `Boss1`, `Boss2`, and `Boss3`, producing 87 enemy or boss instances. Its authored scoring denominator is 125: sixteen chests, ninety-one coin entities with a combined amount of 240, and eighteen potions. Seven checkpoints, one completion cup, and four portals forming two pairs structure the route. No `FakeWall` is authored, and dedicated boss patrol zones support the three major encounters.

The final level changes the visual theme to `ColdCorridor` and presents all three 2D bosses. Their two, three, and four phases create nine full boss phases in one level. The much larger authored economy and the 360-second par value reflect the long sequence of regular encounters, rewards, and boss arenas.

Active Level 1 file

Because `assets/levels/tutorial.ldtk` exists, `GameController::GetLevelPath(1)` selects it before the legacy fallback. This catalog therefore uses the LDtk Tutorial as L2D-01; `assets/levels/level1.lvl` is not the active first campaign level.

14 Fifty Survival3D Waves - Separate Level Catalog

The identifiers W3D-01–W3D-50 describe playable wave content and are **not counted as features**. Let w be the current wave and $t = w - 1$. The loaded balance configuration applies

$$H(w) = 0.55 (1 + 0.018t + 0.0002t^2), \quad D(w) = 0.35 (1 + 0.008t + 0.00008t^2),$$

$$B(w) = \text{round}((5 + 0.65w + 0.012w^2)m_w), \quad I(w) = \max\left(0.5, \frac{1.1}{1 + 0.03t}\right),$$

$$C(w) = \min(120, 10 + \lfloor 0.4w \rfloor).$$

Here $m_{48} = 1.15$ and $m_w = 1$ for every other wave. $H(w)$ and $D(w)$ already include the configured enemy assist multipliers 0.55 and 0.35. Regular-enemy movement speed is

$$S_a(w) = S_{a,0} (1 + \min(0.30, 0.006t)) \times 0.78,$$

where the base HP/speed/damage triples are Riftling 22/6.8/7, Hex Archer 55/4.2/11, and Obsidian Brute 320/2.7/26.

Riftling, Hex Archer, and Obsidian Brute cost 1, 3, and 7 threat respectively. The Swarm–Ranger–Tanker mix is a probability distribution used when selecting the next archetype; it is not a guaranteed final enemy count. The code tests Tanker probability first, Ranger second, and assigns the remaining probability to Swarm. If the selected cost exceeds the remaining threat, the director downgrades to Hex Archer when at least 3 threat remains, otherwise to Riftling. Four gates on the arena sides provide randomized spawn positions. Boss waves set the remaining spawn rule to exactly one boss; the ordinary $B(w)$ value is retained in the table only as a mathematical reference.

Every wave begins with a 3.0-second pre-wave phase. From W3D-02 onward, the player heals 65% of maximum HP, retains at least a 12% maximum-HP shield, and gains 20 ultimate charge. A cleared combat phase displays `WaveClear` for 1.2 seconds before upgrade selection; W3D-50 proceeds to `RunVictory`. Boss summons use pool-headroom guards of 116 active enemies for Brood Warden and Hexeye Artillerist, or 112 for Void Sovereign, rather than the ordinary per-wave active cap.

Chapter evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`; `AppleKnightAdventure/include/Survival3D/Controller/SurvivalController.h`.

Table 4: Director parameters for W3D-01–W3D-50

Wave	Mix or boss	Threat budget or rule	Cap	Interval	HP scale	DMG scale	Primary pressure
W3D-01	100/0/0	6	10	1.100 s	0.550	0.350	Exactly six Riftlings; basic movement and melee tutorial
W3D-02	100/0/0	6	10	1.068 s	0.560	0.353	Same six-unit roster with slightly faster timing and scaling
W3D-03	85/15/0	7	11	1.038 s	0.570	0.356	First probability of a ranged Hex Archer
W3D-04	80/20/0	8	11	1.009 s	0.581	0.359	Greater Ranger probability while Swarm remains dominant
W3D-05	75/25/0	9	12	0.982 s	0.591	0.362	First sub-one-second interval and 25% Ranger weight
W3D-06	85/15/0	9	12	0.957 s	0.602	0.365	Composition respite with higher stats and equal budget
W3D-07	70/30/0	10	12	0.932 s	0.613	0.368	Early projectile pressure reaches 30% Ranger weight
W3D-08	80/20/0	11	13	0.909 s	0.625	0.371	More Swarm probability but greater budget and cap
W3D-09	70/30/0	12	13	0.887 s	0.636	0.374	Final mixed gate before the first boss

Wave	Mix or boss	Threat budget or rule	Cap	Interval	HP scale	DMG scale	Primary pressure
W3D-10	Boss: Brood Warden	1 boss ($B = 13$)	14	0.866 s	0.648	0.377	Two-phase claw boss that summons Riftlings
W3D-11	60/30/10	14	14	0.846 s	0.660	0.381	First probability of the cost-7 Obsidian Brute
W3D-12	55/30/15	15	14	0.827 s	0.672	0.384	Three roles with increased Tanker weight
W3D-13	65/20/15	15	15	0.809 s	0.685	0.388	Swarm-heavy composition with Tanker support
W3D-14	50/35/15	16	15	0.791 s	0.697	0.391	Strong ranged crossfire behind melee bodies
W3D-15	55/25/20	17	16	0.775 s	0.710	0.395	Tanker probability reaches 20%
W3D-16	60/20/20	18	16	0.759 s	0.723	0.398	More encirclement pressure with the same Tanker weight
W3D-17	45/40/15	20	16	0.743 s	0.737	0.402	Projectile-heavy composition with 40% Ranger weight
W3D-18	55/25/20	21	17	0.728 s	0.750	0.406	Sustained melee and Tanker pressure over a larger budget
W3D-19	45/35/20	22	17	0.714 s	0.764	0.409	Combined crossfire and slam pressure before W3D-20

Wave	Mix or boss	Threat budget or rule	Cap	Interval	HP scale	DMG scale	Primary pressure
W3D-20	Boss: Hexeye Artillerist	1 boss ($B = 23$)	18	0.701 s	0.778	0.413	Kiting artillery boss with Archer summons and expanding volleys
W3D-21	45/40/15	24	18	0.688 s	0.792	0.417	Retains high Ranger pressure after the artillery boss
W3D-22	50/30/20	25	18	0.675 s	0.806	0.421	Balanced 50/30/20 three-role test
W3D-23	40/40/20	26	19	0.663 s	0.821	0.425	Equal Swarm and Ranger probabilities with Tankers
W3D-24	55/25/20	28	19	0.651 s	0.836	0.429	Larger Swarm share converts budget into more bodies
W3D-25	45/30/25	29	20	0.640 s	0.851	0.433	Tanker probability rises to 25%
W3D-26	40/45/15	30	20	0.629 s	0.866	0.438	Ranger-dominant projectile-lane test
W3D-27	50/25/25	31	20	0.618 s	0.882	0.442	Melee and Tanker emphasis
W3D-28	45/35/20	33	21	0.608 s	0.897	0.446	Balanced formation with a higher active cap
W3D-29	35/40/25	34	21	0.598 s	0.913	0.450	Elite-heavy gate before Ironroot Colossus
W3D-30	Boss: Ironroot Colossus	1 boss ($B = 35$)	22	0.588 s	0.930	0.455	Slow two-phase slam boss with three then five fissures

Wave	Mix or boss	Threat budget or rule	Cap	Interval	HP scale	DMG scale	Primary pressure
W3D-31	45/35/20	37	22	0.579 s	0.946	0.459	High-budget return to multi-target combat
W3D-32	50/25/25	38	22	0.570 s	0.963	0.464	Close-range and Tanker-oriented encounter
W3D-33	35/45/20	40	23	0.561 s	0.979	0.468	45% Ranger crossfire with Brute obstruction
W3D-34	45/25/30	41	23	0.553 s	0.996	0.473	First 30% Tanker probability
W3D-35	40/35/25	42	24	0.545 s	1.014	0.478	HP scaling exceeds 1.0
W3D-36	50/30/20	44	24	0.537 s	1.031	0.482	Long balanced encounter near the interval floor
W3D-37	35/40/25	45	24	0.529 s	1.049	0.487	Elite-heavy sustain and projectile test
W3D-38	45/25/30	47	25	0.521 s	1.067	0.492	W3D-34 shape with greater budget, cap, and scale
W3D-39	35/40/25	49	25	0.514 s	1.085	0.497	Final high-elite gate before Eclipse Chimera
W3D-40	Boss: Eclipse Chimera	1 boss ($B = 50$)	26	0.507 s	1.103	0.502	Timed Solar melee and Lunar ranged stances
W3D-41	40/35/25	52	26	0.500 s	1.122	0.507	Spawn interval reaches its permanent 0.500-second floor

Wave	Mix or boss	Threat budget or rule	Cap	Interval	HP scale	DMG scale	Primary pressure
W3D-42	45/30/25	53	26	0.500 s	1.141	0.512	Slightly more Swarm probability with the same cap
W3D-43	35/45/20	55	27	0.500 s	1.160	0.517	Endgame Ranger-heavy pressure
W3D-44	45/25/30	57	27	0.500 s	1.179	0.522	Endgame Tanker-heavy pressure
W3D-45	40/35/25	59	28	0.500 s	1.199	0.527	Dense three-role endgame formation
W3D-46	45/35/20	60	28	0.500 s	1.218	0.533	More bodies than a 30%-Tanker composition typically yields
W3D-47	35/40/25	62	28	0.500 s	1.238	0.538	Standard elite-heavy endurance gate
W3D-48	40/30/30	73	29	0.500 s	1.258	0.543	Deliberate $1.15\times$ budget spike and 30% Tanker probability
W3D-49	35/40/25	66	29	0.500 s	1.279	0.549	Budget falls after W3D-48 while stat scaling continues upward
W3D-50	Boss: Void Sovereign	1 boss ($B = 68$)	30	0.500 s	1.299	0.554	Three full HP bars and the final RunVictory condition

14.1 W3D-01 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 100/0/0; threat budget = 6; active cap = 10; spawn interval = 1.100 s; HP scale = 0.550; damage scale = 0.350.

Pressure structure. Only Riftlings can be selected, and each costs one threat, so this wave produces exactly six regular enemies. It introduces camera-relative movement, the basic attack, enemy melee wind-up, arena collision, and the transition from Combat to Wave Clear without ranged interference.

Director behavior. The active cap is above the total six-unit roster, so the principal pacing variable is the 1.100-second spawn interval. No downgrade ambiguity exists because every legal selection costs one threat.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.2 W3D-02 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 100/0/0; threat budget = 6; active cap = 10; spawn interval = 1.068 s; HP scale = 0.560; damage scale = 0.353.

Pressure structure. The roster is again exactly six Riftlings, but HP, damage, and spawn pacing increase slightly. This tests whether the player can repeat the fundamental movement and basic-combat loop before a ranged archetype becomes possible.

Director behavior. Compared with W3D-01, HP scale rises by approximately 0.010, damage scale by 0.003, and interval falls by 0.032 seconds. Cost-one selection still removes all composition uncertainty.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.3 W3D-03 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 85/15/0; threat budget = 7; active cap = 11; spawn interval = 1.038 s; HP scale = 0.570; damage scale = 0.356.

Pressure structure. A Hex Archer becomes possible for the first time. A Ranger selection costs three threat and therefore reduces the eventual entity count compared with seven all-Riffling selections, but it adds a projectile lane and a target-priority decision.

Director behavior. The 85/15/0 values are probabilities, not a guarantee that every generated roster contains an Archer. A seed may legally produce only Rifflings while the configuration still identifies W3D-03 as the first wave in which a Ranger can be selected.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.4 W3D-04 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 80/20/0; threat budget = 8; active cap = 11; spawn interval = 1.009 s; HP scale = 0.581; damage scale = 0.359.

Pressure structure. Ranger probability increases to 20% while Swarm remains the dominant selection. The added threat point supports a longer formation or a different cost combination, and crossfire becomes more likely than in W3D-03.

Director behavior. A Ranger consumes three of eight threat, so the exact number of enemies depends on roll order and any final downgrade. The active cap of 11 remains above the maximum eight cost-one selections.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.5 W3D-05 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 75/25/0; threat budget = 9; active cap = 12; spawn interval = 0.982 s; HP scale = 0.591; damage scale = 0.362.

Pressure structure. One quarter of selection probability belongs to Hex Archer, and the interval falls below one second for the first time. The player must combine melee crowd control with projectile avoidance rather than remaining stationary in the center.

Director behavior. Nine threat can be spent as nine Riftlings, three Archers, or many mixed sequences after cost checks. These are possible cost decompositions, not guaranteed rosters.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.6 W3D-06 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 85/15/0; threat budget = 9; active cap = 12; spawn interval = 0.957 s; HP scale = 0.602; damage scale = 0.365.

Pressure structure. Ranger weight temporarily falls back to 15%, but budget stays at nine while all scaling and pacing continue upward. This is a composition respite rather than a true difficulty reset.

Director behavior. More of the probability mass again converts threat into cost-one Riftlings, so a typical seed may contain more bodies than W3D-05 even though both waves use budget nine.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.7 W3D-07 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 70/30/0; threat budget = 10; active cap = 12; spawn interval = 0.932 s; HP scale = 0.613; damage scale = 0.368.

Pressure structure. Ranger probability reaches 30%, the highest value in the first nine waves. Projectile lanes can overlap with melee pursuit, requiring deliberate target priority and lateral movement.

Director behavior. Cost-three Ranger rolls consume the ten-threat pool faster than Riftlings, so a projectile-heavy seed can contain fewer active bodies without being an easier encounter.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.8 W3D-08 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 80/20/0; threat budget = 11; active cap = 13; spawn interval = 0.909 s; HP scale = 0.625; damage scale = 0.371.

Pressure structure. Ranger weight drops to 20%, but budget and cap increase. A typical seed therefore shifts back toward a larger group of Riftlings while retaining meaningful ranged disruption.

Director behavior. The active cap of 13 is still above the all-Riftling maximum of eleven, so budget and interval determine total pacing rather than simultaneous-cap throttling.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.9 W3D-09 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 70/30/0; threat budget = 12; active cap = 13; spawn interval = 0.887 s; HP scale = 0.636; damage scale = 0.374.

Pressure structure. The final regular wave of chapter one restores 30% Ranger probability and carries the largest pre-boss budget and fastest pre-boss interval. It is the last mixed-mob readiness check before Brood Warden.

Director behavior. No Tanker can be selected yet. Every valid roster is composed from cost-one Riftlings and cost-three Hex Archers, followed by downgrade handling for the final remaining threat.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.10 W3D-10 — Boss: Brood Warden

Boss parameters

One Brood Warden; ordinary cap reference = 14; interval reference = 0.866 s; HP scale = 0.64801; damage scale = 0.377468; effective boss HP = 1425.622; base hit = 9.437; phase-one speed = 2.652.

Mechanics and strategy. Brood Warden pursues until approximately 3.1 units and uses a 0.60-second claw wind-up with hit range 3.8. Its first summon is enabled by the initial 5.5-second special cooldown; later phase-one summons create two Riftlings every 7.5 seconds. At 55% HP it enters a 1.2-second phase transition, increases speed by 15% to approximately 3.050, summons three Riftlings every 5.5 seconds, and reduces claw cooldown from 1.8 to 1.45 seconds.

Progression meaning and caveat. W3D-10 closes chapter one of five. Summoned adds use the boss pool-headroom check rather than the ordinary cap of 14, so the cap shown above is contextual director data, not a strict summon limit.

Source evidence. `assets/survival3d/config/waves.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.11 W3D-11 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 60/30/10; threat budget = 14; active cap = 14; spawn interval = 0.846 s; HP scale = 0.660; damage scale = 0.381.

Pressure structure. Obsidian Brute becomes possible for the first time. One cost-seven Brute can consume half the total threat, while the remaining probabilities still support projectile pressure and a surrounding Riftling screen.

Director behavior. The 10% Tanker value is only a roll probability. A seed with no Brute is valid, while a selected Brute produces fewer total enemies because of its cost.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.12 W3D-12 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 55/30/15; threat budget = 15; active cap = 14; spawn interval = 0.827 s; HP scale = 0.672; damage scale = 0.384.

Pressure structure. Tanker probability increases to 15% while Ranger remains at 30%. The player may need to move around a Brute blocker while prioritizing Archers and preventing Riftling encirclement.

Director behavior. Because Brutes cost seven and Archers cost three, actual entity count can be much lower than the budget number. The cap of 14 limits simultaneous active enemies only if spawn composition and pacing reach it.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.13 W3D-13 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 65/20/15; threat budget = 15; active cap = 15; spawn interval = 0.809 s; HP scale = 0.685; damage scale = 0.388.

Pressure structure. Swarm probability rises to 65%, Ranger falls to 20%, and Tanker remains 15%. Compared with W3D-12, the same budget is more likely to become a larger body count and stronger encirclement pressure.

Director behavior. Active cap increases to 15, equal to the maximum all-Rifling roster, while interval and stat scaling continue to rise.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.14 W3D-14 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 50/35/15; threat budget = 16; active cap = 15; spawn interval = 0.791 s; HP scale = 0.697; damage scale = 0.391.

Pressure structure. Only half of selection probability remains Swarm, while Ranger rises to 35%. The likely tactical shape is ranged crossfire protected by melee bodies and occasional Brute area denial.

Director behavior. The higher-cost probabilities spend sixteen threat in larger chunks. This can reduce body count while increasing projectile and slam complexity.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.15 W3D-15 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 55/25/20; threat budget = 17; active cap = 16; spawn interval = 0.775 s; HP scale = 0.710; damage scale = 0.395.

Pressure structure. Tanker probability reaches 20%. Seventeen threat can fund two Brutes plus a small remainder, but that is only one possible roll sequence; other seeds produce a denser mixed formation.

Director behavior. The cap rises to 16 while the interval falls to 0.775 seconds. Cost-seven selections make single-target burst and slam avoidance increasingly important.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.16 W3D-16 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 60/20/20; threat budget = 18; active cap = 16; spawn interval = 0.759 s; HP scale = 0.723; damage scale = 0.398.

Pressure structure. Ranger probability falls while Swarm rises, with Tanker remaining at 20%. The result is more likely to combine encirclement with Brute area denial than sustained Archer crossfire.

Director behavior. Budget increases by one and pacing accelerates, so the lower Ranger share does not make this a numerical rollback from W3D-15.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.17 W3D-17 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 45/40/15; threat budget = 20; active cap = 16; spawn interval = 0.743 s; HP scale = 0.737; damage scale = 0.402.

Pressure structure. Ranger probability reaches 40%, making this the most projectile-oriented wave so far. Riftlings still body-block, and a Brute can force the player away from otherwise safe firing lanes.

Director behavior. Twenty threat and a 16-enemy cap support a sustained encounter. A high-Ranger roll spends budget quickly but produces overlapping projectile timing.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.18 W3D-18 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 55/25/20; threat budget = 21; active cap = 17; spawn interval = 0.728 s; HP scale = 0.750; damage scale = 0.406.

Pressure structure. The composition returns to a melee- and Tanker-oriented balance, but budget exceeds twenty and cap rises to seventeen. The encounter therefore tests sustained health, cooldown, and area-management resources.

Director behavior. A lower Ranger share changes the tactical shape without reversing the continuing increases in budget, HP, damage, cap, and spawn speed.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.19 W3D-19 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 45/35/20; threat budget = 22; active cap = 17; spawn interval = 0.714 s; HP scale = 0.764; damage scale = 0.409.

Pressure structure. Ranger at 35% and Tanker at 20% combine projectile and slam pressure immediately before the artillery boss. This is the final three-role readiness gate of chapter two.

Director behavior. Higher-cost selections can reduce enemy count but increase simultaneous mechanical demands. The 0.714-second interval gives less recovery between spawns than every prior regular wave.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.20 W3D-20 — Boss: Hexeye Artillerist

Boss parameters

One Hexeye Artillerist; ordinary cap reference = 18; interval reference = 0.701 s; HP scale = 0.77781; damage scale = 0.413308; effective HP = 2644.554; base hit = 7.440; speed = 2.808.

Mechanics and strategy. Hexeye retreats below 8 units, closes beyond 13, and strafes between those distances. It starts a volley within 16 units after a 0.90-second wind-up. Phase one fires three projectiles at speed 9, radius 0.24, splash 0.75, and cooldown 2.6 seconds. Its first two-Archer summon uses the initial 7-second special cooldown; later phase-one summons use 9 seconds. At 50% HP it enters a 1.2-second transition, fires five-shot volleys with a 2.0-second cooldown, and summons two Archers every 7 seconds.

Progression meaning and caveat. W3D-20 closes chapter two. Archer adds are governed by the boss's 116-active headroom check, not the ordinary cap of 18 shown as context.

Source evidence. `assets/survival3d/config/waves.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.21 W3D-21 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 45/40/15; threat budget = 24; active cap = 18; spawn interval = 0.688 s; HP scale = 0.792; damage scale = 0.417.

Pressure structure. Immediately after the artillery boss, Ranger probability remains at 40%. The wave reuses the learned projectile-lane response while adding a larger mixed budget and possible Brute obstruction.

Director behavior. This is a regular probabilistic roster again, not a continuation of Hexeye's fixed add policy. Archer cost remains three threat and Brute cost remains seven.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.22 W3D-22 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 50/30/20; threat budget = 25; active cap = 18; spawn interval = 0.675 s; HP scale = 0.806; damage scale = 0.421.

Pressure structure. A 50/30/20 mix provides a balanced three-role examination. The player must manage encirclement, projectile lanes, and Brute ground slams without one role overwhelmingly defining the encounter.

Director behavior. Budget rises to twenty-five while cap remains eighteen, so the encounter may continue through several spawn cycles even if simultaneous occupancy is limited.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.23 W3D-23 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 40/40/20; threat budget = 26; active cap = 19; spawn interval = 0.663 s; HP scale = 0.821; damage scale = 0.425.

Pressure structure. Swarm and Ranger probabilities are equal at 40%, with Tanker at 20%. Melee occupation and ranged crossfire are equally likely to consume the director's selection opportunities.

Director behavior. Because their threat costs differ, equal selection probability does not imply equal final counts. Cost-one Riftlings can still outnumber cost-three Archers in the completed roster.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.24 W3D-24 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 55/25/20; threat budget = 28; active cap = 19; spawn interval = 0.651 s; HP scale = 0.836; damage scale = 0.429.

Pressure structure. Riftling probability rises to 55% while budget jumps by two. A typical seed can convert more threat into cost-one bodies, creating an area-damage and chain-effect stress test.

Director behavior. The active cap of nineteen can now matter if many Riftlings accumulate faster than the player removes them. Higher-cost selections shorten the roster but add more complex threats.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.25 W3D-25 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 45/30/25; threat budget = 29; active cap = 20; spawn interval = 0.640 s; HP scale = 0.851; damage scale = 0.433.

Pressure structure. Tanker probability reaches 25%. The wave can combine several high-HP, cost-seven threats with Rangers and Swarm, forcing the player to reserve burst for Brutes without ignoring projectile sources.

Director behavior. A quarter Tanker probability does not mean that exactly one quarter of spawned entities are Brutes; their high cost and downgrade behavior significantly change the final count.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.26 W3D-26 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 40/45/15; threat budget = 30; active cap = 20; spawn interval = 0.629 s; HP scale = 0.866; damage scale = 0.438.

Pressure structure. Ranger probability reaches 45%, making projectile lanes the dominant positioning problem. Rifflings still occupy movement space, while the smaller Brute probability can create a durable obstruction in front of the Archers.

Director behavior. Because an Archer costs three threat, the final roster remains seed- and budget-dependent. A selected Brute costs seven threat and may be downgraded when insufficient budget remains.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.27 W3D-27 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 50/25/25; threat budget = 31; active cap = 20; spawn interval = 0.618 s; HP scale = 0.882; damage scale = 0.442.

Pressure structure. Half of the selection weight returns to Swarm, while Tanker probability remains one quarter. The likely shape is close-range occupation around one or more expensive Brutes, with fewer Ranger selections than W3D-26.

Director behavior. Cost-one Rifflings can convert a large fraction of budget into body count. The cap of twenty prevents every possible body from being active simultaneously when the player clears slowly.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.28 W3D-28 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 45/35/20; threat budget = 33; active cap = 21; spawn interval = 0.608 s; HP scale = 0.897; damage scale = 0.446.

Pressure structure. This is a balanced three-role formation with enough Ranger probability to punish stationary play and enough Tanker probability to delay clearing. The active cap increases to twenty-one as budget rises by two.

Director behavior. A larger cap permits more simultaneous threats, but it remains independent of the eventual roster size. Selection, remaining threat, and pool availability still determine each spawn.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.29 W3D-29 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 35/40/25; threat budget = 34; active cap = 21; spawn interval = 0.598 s; HP scale = 0.913; damage scale = 0.450.

Pressure structure. Ranger and Tanker together hold 65% selection probability. This creates the final elite-heavy gate before Ironroot Colossus: Archers cover lanes while Brutes consume burst resources and restrict safe movement.

Director behavior. The roster is still probabilistic, so an unlucky or favourable sequence may differ between runs. Cost and downgrade rules explain the concrete roster produced by any particular seed.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.30 W3D-30 — Boss: Ironroot Colossus

Boss rule and scaled statistics

Exactly one Ironroot Colossus is spawned. Its approximate HP is 4,833.972, base hit is 14.552, and movement speed is 1.833. The ordinary formula would give $B = 35$, cap 22, interval 0.588 s, HP scale 0.930, and damage scale 0.455, but the boss rule replaces the regular roster.

Phase policy. Ironroot is a slow, durable close-range boss built around a telegraphed slam and ground fissures. At approximately 50% HP it enters its second phase and increases the fissure pattern from three to five lines, reducing the amount of safe floor after each tell.

Player response. The player should stop attacking during the slam commitment, move between fissure lanes, and use the recovery interval rather than spending every dash during neutral movement. Its low speed does not make it harmless because the ground pattern controls a large area.

Source evidence. `assets/survival3d/config/waves.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.31 W3D-31 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 45/35/20; threat budget = 37; active cap = 22; spawn interval = 0.579 s; HP scale = 0.946; damage scale = 0.459.

Pressure structure. The run immediately returns from a single slow boss to high-budget multi-target combat. A 45/35/20 mix restores simultaneous encirclement, projectile, and durability demands.

Director behavior. Budget increases beyond the W3D-29 regular value and cap reaches twenty-two. Wave-start recovery is therefore important but does not fully erase poor resource management during W3D-30.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.32 W3D-32 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 50/25/25; threat budget = 38; active cap = 22; spawn interval = 0.570 s; HP scale = 0.963; damage scale = 0.464.

Pressure structure. Swarm and Tanker emphasis shifts the fight toward close range. Riftlings surround and interrupt movement while cost-seven Brutes reward focused damage rather than indiscriminate projectile trading.

Director behavior. A 25% Tanker probability can consume threat quickly, so a seed with several Brutes may contain fewer total bodies than a Riftling-heavy seed despite equal budget.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.33 W3D-33 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 35/45/20; threat budget = 40; active cap = 23; spawn interval = 0.561 s; HP scale = 0.979; damage scale = 0.468.

Pressure structure. Ranger probability returns to 45% while a fifth of selections may be Tankers. Projectile crossfire becomes harder to escape when a Brute or Riftling occupies the intended dodge lane.

Director behavior. Budget reaches forty and cap twenty-three, so the encounter can sustain several spawn cycles. The probability mix remains a selection policy rather than a roster quota.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.34 W3D-34 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 45/25/30; threat budget = 41; active cap = 23; spawn interval = 0.553 s; HP scale = 0.996; damage scale = 0.473.

Pressure structure. Tanker probability reaches 30% for the first time. The wave exchanges some Ranger pressure for expensive, durable Brutes that can hold space while Swarm units close in.

Director behavior. High Tanker cost often reduces final entity count, but each selection consumes more time and concentrated damage. Downgrades near the end of the budget can still add Archers or Riftlings.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.35 W3D-35 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 40/35/25; threat budget = 42; active cap = 24; spawn interval = 0.545 s; HP scale = 1.014; damage scale = 0.478.

Pressure structure. This is the first row whose regular HP multiplier exceeds 1.0. The three-role distribution is balanced enough that no single counter solves the complete encounter.

Director behavior. Budget, cap, and durability all rise together, so a build that relied only on early-wave raw damage begins to lose clear speed. Upgrade synergy becomes materially visible.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.36 W3D-36 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 50/30/20; threat budget = 44; active cap = 24; spawn interval = 0.537 s; HP scale = 1.031; damage scale = 0.482.

Pressure structure. A familiar 50/30/20 balance is revisited at much larger scale. Half-weight Swarm can create many bodies, while Rangers and Tankers punish a player who treats it as a pure crowd wave.

Director behavior. The interval is close to its floor and budget is forty-four, producing a longer sustained encounter. Active cap remains the simultaneous-occupancy limit, not the total-spawn count.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.37 W3D-37 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 35/40/25; threat budget = 45; active cap = 24; spawn interval = 0.529 s; HP scale = 1.049; damage scale = 0.487.

Pressure structure. Ranger and Tanker selection totals 65%, creating an elite-heavy endurance test. The player must avoid projectile attrition while preventing Brutes from consuming the entire damage rotation.

Director behavior. Regular scaling now raises both durability and damage meaningfully above the early run. Pooling and the cap keep the encounter bounded even when clear speed drops.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.38 W3D-38 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 45/25/30; threat budget = 47; active cap = 25; spawn interval = 0.521 s; HP scale = 1.067; damage scale = 0.492.

Pressure structure. W3D-38 repeats W3D-34's Tanker-heavy shape with greater budget, cap, HP, damage, and spawn speed. It therefore provides a direct demonstration of scaling without changing the probability distribution.

Director behavior. The comparison isolates progression variables: the same mix does not imply the same difficulty once formulas and budget have advanced four waves.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.39 W3D-39 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 35/40/25; threat budget = 49; active cap = 25; spawn interval = 0.514 s; HP scale = 1.085; damage scale = 0.497.

Pressure structure. This is the last regular gate before Eclipse Chimera. Elite probability is high and the interval is only 0.014 seconds above its permanent floor, leaving little idle time between spawn opportunities.

Director behavior. The wave should be evaluated together with the resources carried into W3D-40. Spending every ultimate or defensive option to finish W3D-39 can make the boss transition materially harder.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.40 W3D-40 — Boss: Eclipse Chimera

Boss rule and scaled statistics

Exactly one Eclipse Chimera is spawned. Its approximate HP is 7,723.870, base hit is 17.563, and movement speed is 3.120. The ordinary formula would give $B = 50$, cap 26, interval 0.507 s, HP scale 1.103, and damage scale 0.502, but the boss rule replaces the regular roster.

Phase policy. Chimera alternates a Solar melee stance with a Lunar ranged stance. Above half HP the stance interval is approximately eighteen seconds; below half HP it falls to approximately twelve seconds, increasing how often the player must change positioning logic.

Player response. Solar stance rewards spacing and counter timing around close-range commitments. Lunar stance requires lateral movement and projectile reading. Saving mobility for the transition is safer than treating one stance as the complete fight.

Source evidence. `assets/survival3d/config/waves.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.41 W3D-41 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 40/35/25; threat budget = 52; active cap = 26; spawn interval = 0.500 s; HP scale = 1.122; damage scale = 0.507.

Pressure structure. The spawn interval reaches its permanent floor of half a second. The balanced late-game mix tests whether the build can clear, reposition, and defend under continuous director opportunities.

Director behavior. Later waves can still become harder through budget, cap, HP, damage, and mix even though interval no longer decreases. This row is the clearest demonstration of the clamped interval formula.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.42 W3D-42 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 45/30/25; threat budget = 53; active cap = 26; spawn interval = 0.500 s; HP scale = 1.141; damage scale = 0.512.

Pressure structure. Five percentage points move from Ranger to Swarm compared with W3D-41. A typical seed can therefore contain more bodies without removing the durable Tanker component.

Director behavior. Cap and interval remain unchanged while budget and scaling increase. The comparison illustrates that adjacent waves do not need a new mechanic to produce a measurable difficulty change.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.43 W3D-43 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 35/45/20; threat budget = 55; active cap = 27; spawn interval = 0.500 s; HP scale = 1.160; damage scale = 0.517.

Pressure structure. Endgame Ranger probability reaches 45% at the interval floor. The player must create safe lanes rather than remain locked onto a single close target while projectiles arrive from several gates.

Director behavior. Higher budget and cap sustain crossfire for longer than the earlier Ranger-heavy waves. Archer selection remains subject to cost and downgrade behavior.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.44 W3D-44 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 45/25/30; threat budget = 57; active cap = 27; spawn interval = 0.500 s; HP scale = 1.179; damage scale = 0.522.

Pressure structure. The run switches from Ranger-heavy to Tanker-heavy pressure. Thirty-percent Brute probability places greater value on burst, damage amplification, and avoiding overlapping slam zones.

Director behavior. A high-cost-heavy seed can produce fewer entities but more total time under threat. Remaining budget may still be converted into cheaper archetypes after Brute selections.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.45 W3D-45 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 40/35/25; threat budget = 59; active cap = 28; spawn interval = 0.500 s; HP scale = 1.199; damage scale = 0.527.

Pressure structure. The late-game three-role formation is dense and balanced. No role drops below one quarter probability, so positioning, projectile avoidance, and focused damage remain simultaneously relevant.

Director behavior. Cap rises to twenty-eight while budget approaches sixty. Fixed timestep, pooling, and clear pacing become visible engineering requirements rather than hidden optimisations.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.46 W3D-46 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 45/35/20; threat budget = 60; active cap = 28; spawn interval = 0.500 s; HP scale = 1.218; damage scale = 0.533.

Pressure structure. Reducing Tanker probability from thirty or twenty-five percent typically lets more threat convert into cheaper bodies. The wave can therefore feel more crowded even though it is not the highest-budget row.

Director behavior. Composition and threat cost explain why difficulty cannot be inferred from budget alone. The cap remains twenty-eight and is most relevant to a body-heavy seed.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.47 W3D-47 — Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 35/40/25; threat budget = 62; active cap = 28; spawn interval = 0.500 s; HP scale = 1.238; damage scale = 0.538.

Pressure structure. W3D-47 is the standard elite-heavy endurance gate before the deliberate budget spike. Rangers and Brutes together have 65% selection probability while all enemies carry substantial late-run scaling.

Director behavior. The row establishes a baseline for comparing W3D-48 and W3D-49. It uses the normal multiplier of one rather than a special budget modifier.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.48 W3D-48 — Deliberate Budget-Spike Wave

Director parameters

Swarm/Ranger/Tanker mix = 40/30/30; threat budget = 73 after the $1.15\times$ multiplier; active cap = 29; spawn interval = 0.500 s; HP scale = 1.258; damage scale = 0.543.

Pressure structure. This is the largest regular-wave threat budget and shares the highest Tanker probability. The combination produces a designed pre-finale spike rather than merely following the smooth base polynomial.

Director behavior. Without the special multiplier, the rounded base budget would be lower. Budget seventy-three is also greater than W3D-49's sixty-six, even though W3D-49 has higher HP and damage scaling.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.49 W3D-49 — Final Regular Wave

Director parameters

Swarm/Ranger/Tanker mix = 35/40/25; threat budget = 66; active cap = 29; spawn interval = 0.500 s; HP scale = 1.279; damage scale = 0.549.

Pressure structure. Budget falls from the special W3D-48 spike, but enemy HP and damage continue to rise. The final regular wave is still elite-heavy and tests whether the player can preserve resources for the three-bar final boss.

Director behavior. This row demonstrates non-monotonic budget by design: wave number alone does not imply a larger budget than every preceding row. Scaling and mix must be considered separately.

Source evidence. `assets/survival3d/config/waves.json`; `assets/survival3d/config/balance.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`.

14.50 W3D-50 — Final Boss: Void Sovereign

Three complete boss health bars

Void Sovereign has approximately 5,846.445 HP in phase 1, 7,795.260 HP in phase 2, and 10,393.680 HP in phase 3, for approximately 24,035.385 total HP. The ordinary reference row is $B = 68$, cap 30, interval 0.500 s, HP scale 1.299, and damage scale 0.554; the boss rule spawns exactly one Sovereign.

Phase policy. Every transition replaces the depleted bar with a full new bar and expands the attack or summon policy. Earlier apparent lethal events are phase transitions rather than victory. Summons use a pool-headroom guard near 112 active enemies instead of treating the ordinary director cap as a universal summon cap.

Player response. The player must pace ultimate, dash, defensive options, and recovery across three complete phases. The final phase is the only lethal phase that satisfies the run-victory condition; surviving phases one and two is not enough.

Source evidence. `assets/survival3d/config/waves.json`; `AppleKnightAdventure/src/Survival3D/Controller/SurvivalController.cpp`; `AppleKnightAdventure/src/Survival3D/View/SurvivalView.cpp`.

Catalog Conclusion

The project is broader than a conventional 2D platformer. Its implemented surface includes a four-class campaign with local co-op; item, boss, elemental, boon, and core systems; six authored LDK levels; a fifty-wave three-dimensional survival mode; manifest-driven audio; defensive persistence; a layered rendering and loading pipeline; Shop, MapBuilder, and custom content; and an optional validation backend. The catalog derives 130 independent features from implemented runtime or well-defined optional infrastructure while preserving the 56 levels and waves as a separate content set.

The strongest engineering characteristic is subsystem interaction. A Potion affects health, sound, achievements, and persistence. A three-dimensional hit joins an animation event, VFX, spatial audio, and object pooling. A map can move from editor commands through an adapter into a temporary playtest. The resulting catalog therefore describes the project as an integrated game and toolchain rather than a collection of isolated classes.

A Feature Coverage Matrix

Table 5: Coverage of all 130 independent features

Feature range	Domain	Entry count
F001–F008	Startup, interface, preparation, and mode configuration	8
F009–F020	PlayerInputsMovementCollision	12
F021–F040	Four complete two-dimensional skill sets	20
F041–F060	Items, completion, campaign UI, feedback, and MapBuilder entry	20
F061–F075	EnemyBehavior, AI, boons, cores, elements, co-op, and boss arenas	15
F076–F100	Survival3D heroes, skills, AI, director, runtime, presentation, and results	25
F101–F120	Sounds, persistence, OOD, design patterns, and optional services	20
F121–F130	Shop, custom content, MapBuilder, UI states, build, and configuration	10
F001–F130	All independent project features	130

B Level and Wave Coverage Matrix

Table 6: The 56 content identifiers remain separate from the feature total

Identifier range	Content	Count
L2D-01–L2D-06	Six complete 2D campaign levels with individually authored geometry, encounters, interactions, and completion state	6
W3D-01–W3D-50	Fifty configured Survival3D waves, including five milestone bosses and the final RunVictory state	50
L2D/W3D	Independent level and wave content	56

C Source-Evidence Index

Table 7: Principal evidence locations by subsystem

Subsystem	Principal implementation	Principal data or interface
2D runtime	src/Controller/GameController.cpp	include/Controller/GameController.h
Input	src/Controller/InputController.cpp	include/Controller/InputController.h
Player and skills	src/Model/Player.cpp; four *SkillSet.cpp files	include/Model/CharacterSkillSet.h
Enemy and boss AI	src/Model/Enemy.cpp; Boss.cpp; Boss1/2/3.cpp	include/Model/Enemy.h; Boss.h
Items and progression	src/Factories/ItemFactory.cpp; SaveManager.cpp	include/Utils/Types.h; SaveManager.h
Elements, boons, cores	src/Systems/ElementalSystem.cpp; BuffSystem.cpp; CoreSystem.cpp	Matching headers in include/Systems
2D level data	assets/levels/tutorial.ldtk; lvl2.ldtk through lvl6.ldtk	src/Factories/LevelSourceAdapter.cpp
Survival3D	src/Survival3D/Controller/SurvivalController.cpp	include/Survival3D/Model/SurvivalTypes.h
3D presentation	src/Survival3D/View/SurvivalView.cpp; animation and VFX folders	GLB assets and skill VFX packages
Wave configuration	assets/survival3d/config/waves.json	balance.json; services.json
Audio	src/Systems/SoundManager.cpp	assets/sounds/audio_manifest.json
MapBuilder	src/Controller/MapBuilderController.cpp	src/View/MapBuilderView.cpp; src/Model/Command.cpp

Subsystem	Principal implementation	Principal data or interface
Shop	src/Controller/ShopController.cpp	src/View/ShopView.cpp; src/Model/SaveManager.cpp
Backend	backend/survival3d/SurvivalServerCore.cpp	backend/survival3d/main.cpp
Build	CMakeLists.txt	C++17, asset sync, and optional server target

All relative paths in this index are rooted at `AppleKnightAdventure/`. Individual feature entries provide more specific evidence paths for the implementation described in the catalog.